Heaven's Light is Our Guide

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY, BANGLADESH

QFlow: A Fixed-Point FPGA Accelerator for Adaptive Key-Resource-Aware Route-Candidate Evaluation in Trusted-Relay QKD Networks

A thesis submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Computer Science & Engineering

Author

Shahriar Rizvi

Roll No. 2003104, Section B, Series 2020

Supervised by

Dr. Md. Nazrul Islam Mondal

Professor

Department of Computer Science & Engineering

Rajshahi University of Engineering & Technology

August 2026

ACKNOWLEDGEMENT

All praise and gratitude are due to Almighty Allah for granting me the strength, patience, and opportunity to complete this research.

I express my sincere gratitude to my supervisor, **Dr. Md. Nazrul Islam Mondal**, Professor, Department of Computer Science & Engineering, Rajshahi University of Engineering & Technology. His guidance, careful questions, and encouragement shaped the research objectives and helped me connect the mathematical design with reproducible hardware experiments.

I am thankful to the faculty and staff of the Department of Computer Science & Engineering, RUET, for the academic foundation and research environment that made this work possible. I also acknowledge the open-source communities behind Python, Icarus Verilog, Yosys, OpenROAD, and ngspice, as well as the researchers whose published work provided the technical context for this study.

Finally, I offer heartfelt thanks to my parents, family, friends, and classmates for their patience, prayers, and constant support throughout my undergraduate study.

August 2026 — Shahriar Rizvi

RUET, Rajshahi — Roll No. 2003104

Heaven's Light is Our Guide

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY, BANGLADESH

CERTIFICATE

This is to certify that the thesis entitled ***"QFlow: A Fixed-Point FPGA Accelerator for Adaptive Key-Resource-Aware Route-Candidate Evaluation in Trusted-Relay QKD Networks"****, submitted by* ***Shahriar Rizvi, Roll No. 2003104****, in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Computer Science & Engineering at Rajshahi University of Engineering & Technology, is a record of the candidate's own work carried out under my supervision. To the best of my knowledge, this work has not been submitted, in whole or in part, for the award of any other degree or diploma.*

Supervisor

Dr. Md. Nazrul Islam Mondal
Professor
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology
Rajshahi-6204, Bangladesh

External Examiner

Name: _____
Designation: _____
Institution: _____

Date: _____

DECLARATION

I hereby declare that the work presented in this thesis is original and was conducted by me under the supervision of **Dr. Md. Nazrul Islam Mondal**, Professor, Department of Computer Science & Engineering, Rajshahi University of Engineering & Technology. This thesis has not been submitted, in whole or in part, to this or any other institution for the award of another degree or diploma.

All external ideas, publications, software tools, and data sources used in this work have been acknowledged. Physical measurements, post-route implementation results, RTL and software simulations, analytical calculations, and supporting physical-design results are identified according to their respective evidence types.

The repository materials, scripts, fixed seeds, model records, verification vectors, implementation reports, and physical replay summaries are maintained to support reproducibility and independent examination of the reported results.

Date: August 2026

Place: RUET, Rajshahi

Shahriar Rizvi

Roll No. 2003104

ABSTRACT

Quantum key distribution (QKD) networks require routing decisions that account for finite link-local key resources and changing conditions. This thesis presents **QFlow**, a deterministic fixed-point FPGA accelerator for evaluating externally generated route candidates in trusted-relay QKD networks. The upstream KMS/SDN control plane generates candidates, evaluates hard constraints, and supplies each candidate's validity bit and bounded descriptors; authorization and route installation also remain outside the FPGA.

Four features are quantized into 256 states with four profile actions. Tabular Q-learning is performed offline, and the mapping is deployed as a 256-entry two-bit policy ROM with minimum-dwell control. Training uses four coefficients to construct profile-dependent path cost and three Tchebycheff ratios. During physical replay, path cost is precomputed and common across profiles, so H2–H4 deploy only the ratio projection. Physical exactness therefore validates that projection, not on-chip execution of the complete training action. Five independent configurations comprise a feasible minimum-hop baseline (H0), fixed key-aware baseline (H1), fixed-profile **QFlow** (H2), rule-adaptive selection (H3), and learned profile selection (H4).

The experiment uses minimum key occupancy, bounded route quality, offered load, and utilization imbalance. Route quality is an abstract input, not quantum coherence of post-processed classical keys. The network model separately represents key-pool generation, consumption, status, utilization, freshness policy, and health.

Verification combined a fixed-point reference, exhaustive tests, cycle-accurate RTL simulation, independent Xilinx ISE implementations, and physical replay on a Nexys 3 Spartan-6 FPGA. Across five builds, 420 records and 3,780 checked fields produced zero mismatches; every configuration exceeded 100 MHz. H4 achieved 102.020 MHz and mean accepted-request-to-done kernel latency of approximately 2.048 μs. Relative to H3, H4 added 0.986% mean latency, reduced maximum frequency by 0.612%, and produced 24 rather than 44 profile transitions, with fewer LUTs but more registers and occupied slices. Reproducible paired-row confidence intervals crossed zero, with sign-test p-values 0.2188 and 0.4531; route-quality superiority was not established.

Supporting synthesis and OpenROAD results concern isolated kernels, not an integrated ASIC. The contribution is a bit-exact, microsecond-scale adaptive candidate evaluator with explicit semantic and measurement limits.

Keywords: quantum key distribution network, FPGA accelerator, route-candidate evaluation, fixed-point arithmetic, reinforcement learning, policy ROM, trusted relay.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ASIC	Application-Specific Integrated Circuit
BRAM	Block Random-Access Memory
CI	Confidence Interval
DRC	Design Rule Check
DRL	Deep Reinforcement Learning
DSP	Digital Signal Processing slice
FPGA	Field-Programmable Gate Array
FSM	Finite-State Machine
H0–H4	Hardware evaluation configurations
ISE	Xilinx Integrated Software Environment
KMS	Key Management System
LUT	Look-Up Table / FPGA logic element, according to context
LVS	Layout Versus Schematic
QBER	Quantum Bit Error Rate
QKD	Quantum Key Distribution
RL	Reinforcement Learning
ROM	Read-Only Memory
RTL	Register-Transfer Level
SDN	Software-Defined Networking
SKAG	Stochastic Key Availability Graph (repository kernel identifier)
UART	Universal Asynchronous Receiver/Transmitter

LIST OF SYMBOLS

Symbol	Meaning
$G = (V, E)$	Directed trusted-relay QKD network graph
$K_{ij}(t)$	Available classical key units in link pool (i, j) at time t
$K_{ij}^{\max}$	Configured capacity of key pool (i, j)
$\lambda_{ij}(t)$	Effective key-generation rate of link (i, j)
$C_{ij}(t)$	Key-consumption rate of link (i, j)
$X_{ij}(t)$	Key units expired or revoked by policy
$o_{ij}(t)$	Normalized occupancy of key pool (i, j)
$q_{ij}(t)$	Externally measured QBER or health indicator for link (i, j)
α_{fiber}	Physical fibre attenuation coefficient
$\mathcal{R}_{sd}$	Externally generated candidate set for source s and destination d
$\mathcal{R}_i$	Candidate route with index i
$\boldsymbol{\alpha}^{(k)}$	Scoring-coefficient vector for profile Π_k
$\boldsymbol{\lambda}_{\text{TCH}}^{(k)}$	Tchebycheff objective-weight vector for profile Π_k
Π_k	Scoring profile k, where $k \in \{0, 1, 2, 3\}$
N_c	Number of feasible candidates in the offered set
$\boldsymbol{z}^\star$	Componentwise ideal point over feasible candidates
$N_{i,j}$	UNORM16 normalized penalty for candidate i, objective j
$S_i^{(k)}$	18-bit Tchebycheff score of candidate $\mathcal{R}_i$ under profile k
(x_K, x_Q, x_L, x_I)	Occupancy, route-quality, load, and imbalance controller inputs
$z[n]$	Encoded controller state in $\{0, \ldots, 255\}$
$a[n]$	Selected profile action in $\{0, 1, 2, 3\}$
$\pi(z)$	Offline-trained policy-ROM mapping from state to profile
r_t^{train}	Reward used in the offline Q-learning target
r_t^{eval}	Common evaluation reward reported for completed controllers
T_{clk}	FPGA clock period
L	Kernel latency in cycles or time
$f_{\max}$	Post-route maximum operating frequency

CHAPTER 1
INTRODUCTION

1.1 Introduction

Trusted-relay quantum key distribution (QKD) networks must coordinate quantum-link resources with conventional network control. This thesis addresses one recurring control-plane computation: evaluating a bounded set of admissible route candidates from key-resource and network-state metadata. It develops **QFlow**, a deterministic fixed-point FPGA accelerator, and evaluates five independently implemented hardware configurations under a common physical measurement contract.

1.2 Motivation

1.2.1 QKD links and trusted-relay networking

QKD enables two endpoints to establish shared secret material by combining quantum transmission with authenticated classical communication. After sifting, parameter estimation, error correction, and privacy amplification, the usable keys are classical data managed by a key management system (KMS). A network service therefore depends not only on the quantum link but also on key storage, authorization, consumption, replenishment, monitoring, and failure handling [1, 2].

Direct QKD links do not necessarily connect every communicating pair. Trusted-relay networks extend coverage by forwarding key material or coordinating secure segments through intermediate trusted sites. Operational demonstrations such as MadQCI and a carrier-scale network show the importance of integrating QKD devices with KMS and software-defined networking (SDN) functions [3, 4]. Route selection becomes a control problem because candidate paths can differ in distance, hop count, available key material, key-generation rate, utilization, and operational health.

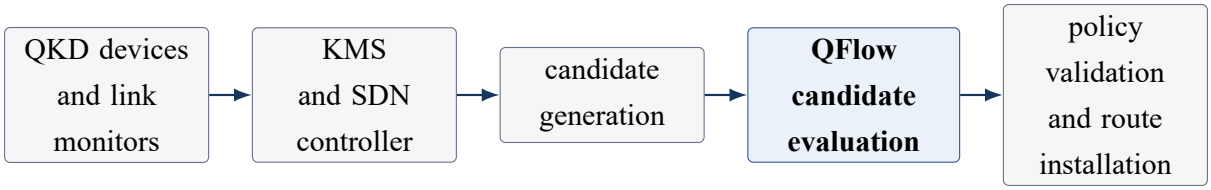


Figure 1.1. System context of **QFlow**. Candidate generation, hard-eligibility computation, and route installation are control-plane functions outside the FPGA kernel.

1.2.2 Key-pool-aware routing control

Post-processed keys are finite classical resources. For a directed link, the KMS can report pool occupancy, effective key-generation rate, consumption, link status, utilization, and externally measured health indicators such as QBER-derived status. An optional age or expiry field represents KMS or security policy; it is not a quantum coherence property of stored bits. A route that is short in distance may be undesirable when one of its key pools is depleted, while a slightly longer route may provide greater reserve or replenishment capacity. Recent work consequently studies dynamic key-aware, multipath, joint-resource, and reinforcement-learning approaches [5, 6, 7, 8, 9].

Two decision layers are useful. Hard eligibility first excludes a route whose links are unavailable, under-provisioned, or administratively invalid. Soft preference then ranks the remaining routes according to a selected scoring profile. Keeping these layers separate prevents a high score from making an inadmissible route appear usable.

1.2.3 Route-candidate evaluation as the selected problem

Full graph routing includes topology acquisition, constraint processing, candidate generation, global optimization, security validation, and route installation. Mapping that entire workflow to a small FPGA would mix irregular control tasks with a compact recurring arithmetic task. This thesis instead accelerates the latter. For source s and destination d, software supplies

$$\mathcal{R}_{sd} = \{\mathcal{R}_0, \mathcal{R}_1, \ldots, \mathcal{R}_{m-1}\}. \tag{1.1}$$

Each candidate $\mathcal{R}_i$ has a fixed-width descriptor containing an upstream validity result, path cost, a bounded route-quality code, utilization, hop count, and a deterministic identifier. **QFlow** selects a scoring profile, gates candidates using the supplied validity bits, evaluates the remaining descriptors, and returns the chosen identifier, score, profile, and status.

This boundary makes exact verification possible. The same descriptor and fixed-point contract can be evaluated in software, RTL simulation, and physical hardware. It also reflects a practical heterogeneous design in which the CPU retains graph and policy responsibilities while the FPGA offloads repeated deterministic evaluation.

1.2.4 Why deterministic hardware acceleration is relevant

A candidate evaluator applies the same bounded operations to every request: quantization, read-only policy lookup, supplied-validity gating, weighted fixed-point scoring, comparison, and deterministic tie-breaking. These operations are suitable for dedicated pipelines and finite-state control. FPGA implementation provides cycle-level predictability, explicit numeric widths, and reconfiguration as the controller or scoring profiles evolve.

The present research stage also benefits from avoiding ASIC fabrication and from implementing all comparison configurations on the same device. CPU software remains essential for orchestration, candidate generation, offline learning, and irregular tasks. The platform choice is therefore a partitioning decision rather than a claim that one platform is universally superior.

1.3 Research Gap

The literature contains sophisticated software studies of QKD routing and resource allocation, including adaptive and reinforcement-learning methods. Separately, FPGA-based QKD research has demonstrated high-throughput reconciliation, post-processing, and synchronization [10, 11, 12, 13]. Within the documented literature review, no directly comparable same-board implementation of the evaluated candidate-selection function was identified. The gap is therefore an implementation and evidence gap: a compact, adaptive, fixed-point candidate evaluator has not been compared through independent physical configurations with a common interface, exact reference agreement, and post-route timing and resource evidence.

1.4 System Scope and Boundary

QFlow is a route-candidate evaluator. It consumes candidate descriptors and network-state metadata that have already been validated and quantized at the software/hardware boundary. It supports fixed, rule-adaptive, and offline-learned profile selection. The learned configuration performs online inference from a read-only policy; Q-values are not updated during FPGA execution.

Global topology management, candidate generation, KMS operation, SDN orchestration, security-policy enforcement, secret-key delivery, and route installation remain in software or the control plane. The FPGA is neither a complete graph router nor a QKD physical-layer device. The physical experiment measures the accepted-request-to-done kernel boundary; it does not measure complete network provisioning latency.

1.5 Problem Statement

Given an externally generated candidate set $\mathcal{R}_{sd}$ and quantized key-resource and network-state metadata, the research problem is to design a bounded hardware kernel that:

1. excludes candidates whose upstream validity bits report failed hard constraints;
2. selects a fixed or adaptive scoring profile deterministically;
3. evaluates feasible candidates using reproducible fixed-point arithmetic;
4. returns a stable result under a synchronous request/done protocol; and

5. is physically implementable on a resource-constrained FPGA with measurable timing, resource, exactness, and adaptive-behavior trade-offs.

1.6 Research Objectives

The objectives are to:

1. formulate a classical key-resource model based on pool occupancy, generation rate, consumption, status, utilization, health indicators, and policy-controlled freshness;
2. define a fixed-point candidate-scoring contract with distinct upstream hard eligibility, FPGA validity gating, and soft preference;
3. develop fixed, rule-adaptive, and offline-learned profile controllers;
4. implement five independent FPGA configurations under one external contract;
5. verify reference-to-RTL and reference-to-board exactness;
6. measure post-route timing, kernel latency, resources, and profile behavior; and
7. interpret route-quality and platform trade-offs without crossing evidence boundaries.

1.7 Research Questions

RQ1. Can all five independently implemented configurations meet a 100 MHz post-route target on the selected FPGA?

RQ2. Do physical outputs match the fixed-point reference exactly under a common replay and output schema?

RQ3. What latency, maximum-frequency, and resource trade-offs arise across the fixed and adaptive configurations?

RQ4. Does the offline-learned controller exhibit distinct and controlled profile behavior relative to the rule-adaptive controller?

RQ5. Do the held-out physical comparisons establish statistically significant route-quality improvement for the learned configuration?

1.8 Research Hypotheses

RH1. Each independent hardware evaluation configuration will meet the 100 MHz post-route timing target.

RH2. Every checked physical result field will match the registered fixed-point reference.

RH3. The reinforcement-learned adaptive configuration will retain microsecond-scale kernel latency with controlled implementation overhead.

RH4. The reinforcement-learned adaptive configuration with minimum-dwell control will produce fewer profile transitions than the rule-adaptive configuration on the common replay.

RH5. The reinforcement-learned adaptive configuration will achieve statistically significant route-quality improvement over fixed-profile and rule-adaptive alternatives on the held-out physical pairs.

1.9 Contributions

The thesis makes the following contributions:

1. a classical key-resource model for candidate evaluation based on pool occupancy, generation rate, consumption, operational status, policy freshness, and externally measured health indicators;
2. a deterministic fixed-point candidate-evaluation contract that separates upstream hard-eligibility computation from FPGA validity gating and profile-conditioned preference;
3. an adaptive controller with four radix-4 features, 256 states, four full software-training profile actions, offline tabular Q-learning, and a 256-entry two-bit policy ROM whose physical candidate shell deploys the profiles' objective ratios with minimum-dwell control;
4. five independent FPGA implementations with a common interface and measurement boundary;
5. a same-board campaign comprising 420 physical records, 3,780 checked fields, and exact reference agreement; and
6. a complete engineering comparison of timing, latency, resources, adaptive behavior, route-quality statistics, and stage-specific platform implications.

1.10 Practical Significance

In a prospective KMS/SDN deployment, candidate evaluation may recur whenever requests arrive or resource state changes. Offloading this fixed-width calculation can provide predictable control latency and a bit-exact implementation shared across tests and deployments. Reprogrammable policy and profile storage also allows the accelerator to evolve without replacing the surrounding control plane.

1.11 User Requirements and Broader Considerations

The engineering users of the proposed kernel are network-control and security software developers. Their principal requirements are deterministic requests and responses, explicit numeric formats, visible failure status, reproducible profiles, and a software fallback. A deployment must authenticate the host/FPGA interface, validate inputs, protect policy artifacts, log configuration identity, and retain the controller's authority over final route installation.

The work does not process human-subject data. Its societal relevance lies in supporting reproducible research on secure-network control rather than promising a complete commercial service. Hardware safety is limited to normal laboratory electrical and electrostatic-discharge practice. Environmental benefit is not quantified because no activity-calibrated power measurement or life-cycle study was performed; reconfigurability can, however, extend prototype usefulness by allowing new bitstreams on the same device.

1.12 Thesis Organization

Chapter 2 reviews trusted-relay QKD networking, key-resource control, multi-objective scoring, reinforcement learning, fixed-point arithmetic, platform alternatives, and related work. Chapter 3 defines the system model, scoring profiles, hardware configurations, controller, RTL, verification, FPGA experiment, statistical method, and supporting VLSI method. Chapter 4 presents and discusses the software, RTL, post-route, physical replay, adaptive-behavior, route-quality, platform, and VLSI results. Chapter 5 summarizes the findings, contributions, limitations, and future work. The appendices provide reproducibility records, fixed-point and protocol details, additional results, and experimental-scope notes.

1.13 Conclusion

The selected problem is a bounded, recurring route-candidate evaluation task within a larger KMS/SDN workflow. The research therefore concentrates on fixed-point correctness, deterministic control, physical FPGA implementation, and transparent trade-offs. The next chapter establishes the technical and literature foundations for that design.

CHAPTER 2
BACKGROUND AND LITERATURE REVIEW

2.1 Introduction

This chapter develops the technical background for key-resource-aware candidate evaluation and positions **QFlow** within QKD networking, adaptive control, and reconfigurable computing. It also defines a comparison taxonomy that separates same-function hardware evidence from network simulations and component-level FPGA studies.

2.2 Trusted-Relay QKD Networks

QKD establishes secret material across a quantum channel and an authenticated classical channel. Networking adds key management, trusted relay operation, application interfaces, routing, monitoring, and recovery. Surveys describe QKD networks as layered systems in which physical links generate secret material and classical control software manages its use [1, 2]. In a trusted-relay architecture, intermediate sites are trusted with the relevant key-handling operations. The trust assumption differs from entanglement-repeater architectures and must be part of the security policy.

Once link post-processing completes, the KMS stores classical key data and metadata. For link (i, j), useful control quantities include available key units $K_{ij}(t)$, pool capacity $K_{ij}^{\max}$, effective generation rate $\lambda_{ij}(t)$, consumption rate $C_{ij}(t)$, operational status, utilization, and health indicators derived from QBER or other monitored values. The physical fibre model may use attenuation coefficient α_{fiber}, but this quantity is distinct from controller scoring coefficients.

2.3 Key Pools and Resource Evolution

A discrete-time key-pool balance can be written as

$$K_{ij}[n+1] = \text{clip}_{[0,K_{ij}^{\max}]}\left(K_{ij}[n] + G_{ij}[n] - C_{ij}[n] - X_{ij}[n]\right), \tag{2.1}$$

where $G_{ij}[n]$ is newly admitted secret material, $C_{ij}[n]$ is consumed material, and $X_{ij}[n]$ is material removed through expiry, revocation, alarms, or administrative action. Every term must use the same unit, such as bits or fixed key blocks. A normalized occupancy is

$$o_{ij}[n] = \frac{K_{ij}[n]}{K_{ij}^{\max}}, \qquad 0 \le o_{ij}[n] \le 1. \tag{2.2}$$

Generation and consumption rates convey information that occupancy alone cannot. Two links with equal current occupancy can have different future risk if one is replenishing quickly

and the other serves a high request load. A useful candidate descriptor therefore combines present reserve with bounded rate, load, and health information.

Freshness, cryptoperiod, or expiry is a KMS/security-policy property. A policy may invalidate material after a configured time, assign a soft preference to newer authorized batches, or revoke material after an alarm. These operations concern the management of classical keys; they are not quantum-state evolution in memory.

2.4 Hard Eligibility and Soft Preference

For a request requiring $D[n]$ key units, a directed link can be represented by the hard eligibility indicator

$$\chi_{ij}[n; D] = \mathbf{1}\{\text{status}_{ij} = \text{UP}\}\mathbf{1}\{K_{ij}[n] \geq D[n]\}\mathbf{1}\{q_{ij}[n] \leq q_{\text{abort}}\}\mathbf{1}\{\mathcal{M}_{ij}[n] \text{ is valid}\}, \quad (2.3)$$

where $\mathcal{M}_{ij}$ collects authenticated administrative metadata. Candidate $\mathcal{R}_i$ is feasible when every constituent link is eligible:

$$v_i[n] = \prod_{(u,v)\in\mathcal{R}_i} \chi_{uv}[n; D]. \quad (2.4)$$

Hard checks protect invariants. Soft preference is applied only after feasibility is known and expresses engineering priorities among admissible candidates. This separation avoids using a weighted score as a substitute for security or resource constraints. In the intended system, the upstream KMS/SDN controller evaluates Equations (2.3)–(2.4). The FPGA receives the resulting v_i bit and uses it only to exclude invalid candidates from range formation and winner selection; it does not independently read link-level KMS records or recompute these equations.

2.4.1 Concrete trusted-relay example

Consider five trusted sites A–E. A request from A to E requires four key units on every traversed link. The control plane supplies two simple routes, $\mathcal{R}_0 = A \to B \to E$ and $\mathcal{R}_1 = A \to C \to D \to E$. The link-local pools and monitored values at one decision instant are shown in Table 2.1. Key units are illustrative fixed blocks; the example does not assume that key bits are copied end to end or that stored classical keys possess a decaying quantum state.

Table 2.1. Five-site trusted-relay example for a four-key-unit request. Hard eligibility uses status, available pool, and the independently defined QBER alarm.

Link	Pool	Capacity	QBER	Util.	Status and interpretation
$A \to B$	9	16	0.021	0.25	Up; enough key units
$B \to E$	5	16	0.028	0.31	Up; bottleneck of $\mathcal{R}_0$
$A \to C$	12	16	0.019	0.18	Up; enough key units
$C \to D$	3	16	0.024	0.16	Up, but insufficient for the request
$D \to E$	10	16	0.026	0.20	Up; enough key units

Figure 2.1 makes the ordering explicit: eligibility is formed from link-local pools before any profile-conditioned comparison, and the single insufficient $C \to D$ pool excludes the entire second route.

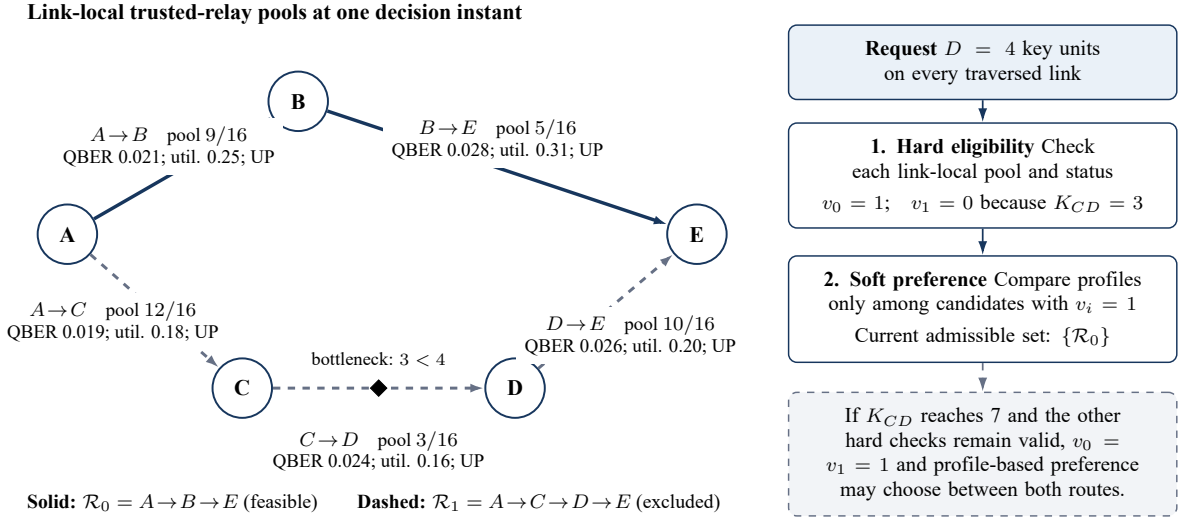


Figure 2.1. Trusted-relay candidate example. Solid links form $\mathcal{R}_0 = A \to B \to E$, which is feasible for a four-key-unit request; dashed links form $\mathcal{R}_1 = A \to C \to D \to E$, which is excluded because the diamond-marked $C \to D$ bottleneck contains only three units. Pools are link-local, and soft profile-based comparison occurs only after upstream hard eligibility is satisfied.

Initially $v_0 = 1$ and $v_1 = 0$, so the evaluator must return $\mathcal{R}_0$ even if $\mathcal{R}_1$ would have a more attractive soft score. If the $C \to D$ pool later reaches seven units while all hard checks remain valid, both candidates become feasible. In the software training environment, a scarcity profile can change the key-count and key-rate coefficients used to form path cost and can also emphasize utilization. In the physical ratio-only projection, the profile registered as Scarcity protection uses ratio $(1, 1, 2)$, so its strongest direct arithmetic emphasis is the supplied utilization objective. A path-cost- emphasis profile may instead prefer the two-hop $\mathcal{R}_0$. Thus feasibility answers "may this route be used?", whereas the profile answers "which permitted route best expresses the current operating preference?" No finite score can override $v_i = 0$.

2.5 Candidate-Route Evaluation

Candidate-generation algorithms may use shortest paths, constrained path search, multipath methods, optimization, or policy filters. **QFlow** receives the resulting bounded set $\mathcal{R}_{sd}$ rather than traversing the complete graph. Each candidate descriptor in the completed FPGA experiment contains a validity bit, precomputed path cost, bounded bottleneck route-quality code, utilization objective, hop count, and candidate slot. Candidate evaluation then becomes a regular reduction: compute a score for every feasible descriptor and retain the best candidate under a deterministic tie-break rule. The broader network-control model may use occupancy, generation, consumption, freshness, status, utilization, and external health indicators to generate these descriptors, but those upstream quantities must not be conflated with the executed controller ports.

The distinction between generation and evaluation is important for both science and implementation. Network-level papers may assess blocking, topology robustness, or global path quality, while an evaluator implementation may assess arithmetic exactness, cycles, post-route timing, and resources. The measurements are related but not interchangeable.

2.6 Multi-Objective Scoring

Key-aware selection combines objectives that have different units. The registered candidate evaluator minimizes three quantities for every feasible candidate: precomputed path cost c_i, route-quality deficit $d_i = 65535 - q_i$, and utilization imbalance u_i. Let

$$\boldsymbol{f}_i = [c_i, d_i, u_i]^{\mathsf{T}}. \tag{2.5}$$

For the currently valid candidates, the ideal point $\boldsymbol{z}^\star = [\min_i c_i, \min_i d_i, \min_i u_i]^{\mathsf{T}}$ contains the best observed value of each objective. It is a reference vector; it need not correspond to one physical route. Each nonnegative distance from the ideal is range-normalized to an unsigned 16-bit penalty $N_{i,j} \in [0, 65535]$. Consequently, zero is ideal and a lower value is better.

Tchebycheff decomposition retains the worst weighted departure from that ideal [14]:

$$S_i^{(k)} = \max_{j \in \{c,q,u\}} \left\{ \lambda_j^{(k)} N_{i,j} \right\}. \tag{2.6}$$

The candidate having the smallest $S_i^{(k)}$ wins. This "minimize the maximum weighted regret" interpretation is different from summing all objectives: a very poor component cannot be hidden completely by two good components. The weight ratios can remain integer because multiplying every $\lambda_j^{(k)}$ by a common positive constant does not change the argmin. Multi-objective routing surveys and restricted-shortest-path theory motivate explicit objective treatment but do not imply that one method is universally best [15, 16, 17].

Each profile contains four half-unit coefficients $\boldsymbol{\alpha}^{(k)}$ and three Tchebycheff ratios. Offline

training uses the full action: the alpha coefficients form a profile-dependent path cost, and the ratios weight the three normalized objectives. In the completed physical shell, candidate path cost arrives precomputed and remains common across profiles; the candidate evaluator consumes only the profile word's lower six ratio bits. Therefore the physical H2–H4 comparison validates a ratio-only projection that changes the emphasis among supplied path cost, route-quality deficit, and utilization. It does not establish equivalence with the complete software action or recompute an alpha-weighted graph path inside the FPGA.

2.6.1 Worked profile-scoring example

Table 2.2 gives two feasible descriptors using the exact numeric contract. UNORM16 values are obtained by round-half-up multiplication by 65,535; UQ16.16 path cost is obtained by round-half-up multiplication by 65,536.

Table 2.2. Inputs to the worked candidate-scoring example.

Route	Cost	Cost code	Quality	Quality code	Util. code	Hops/slot
$\mathcal{R}_0$	1.5	98,304	0.95	62,258	16,384	3 / 0
$\mathcal{R}_1$	2.0	131,072	0.97	63,569	8,192	4 / 1

Cost favours $\mathcal{R}_0$, whereas quality and utilization favour $\mathcal{R}_1$. After the registered finite-range normalization, the three minimization-penalty vectors are

$$\boldsymbol{N}_0 = [0, 65485, 65527], \qquad \boldsymbol{N}_1 = [65533, 0, 0]. \tag{2.7}$$

For balanced profile Π_0, whose hardware ratio is $(2, 2, 1)$,

$$S_0^{(0)} = \max(0, 2 \cdot 65485, 65527) = 130970, \tag{2.8}$$
$$S_1^{(0)} = \max(2 \cdot 65533, 0, 0) = 131066, \tag{2.9}$$

so $\mathcal{R}_0$ wins by 96 score units. For Π_1, registered as Scarcity protection, the deployed ratio is $(1, 1, 2)$ and therefore emphasizes the utilization penalty most strongly:

$$S_0^{(1)} = \max(0, 65485, 2 \cdot 65527) = 131054, \tag{2.10}$$
$$S_1^{(1)} = \max(65533, 0, 0) = 65533, \tag{2.11}$$

so $\mathcal{R}_1$ wins. This example concerns the physical ratio-only projection; the software training action may also change candidate costs through $\boldsymbol{\alpha}^{(1)}$. If two scores tie, comparison continues lexicographically by raw path cost, hop count, and candidate slot, guaranteeing one reproducible result.

2.6.2 Worked fixed-point calculation

For any objective with candidate minimum m and maximum M, the implemented normalization is

$$N(x) = \begin{cases} 0, & M = m, \\ \text{round}_{\text{half up}}\left(\dfrac{(x-m)65535}{(M-m)+1}\right), & M > m. \end{cases} \tag{2.12}$$

The denominator offset prevents the maximum finite difference from producing an out-of-range code. For $\mathcal{R}_1$'s cost, $x - m = 131072 - 98304 = 32768$, hence

$$N_c(\mathcal{R}_1) = \text{round}_{\text{half up}}\left(\frac{32768 \times 65535}{32768 + 1}\right) = 65533. \tag{2.13}$$

The intermediate product is held at sufficient width, the divider returns the rounded unsigned result, and the final value saturates to $[0, 65535]$. A weight ratio of two produces $2 \times 65533 = 131066$, which fits the 18-bit score field. No binary-to-real conversion is used in the comparison: integer 131066 is compared directly with 130970. Saturation handles values above the declared field maximum; it never wraps a large penalty into an apparently favourable small value.

2.7 Offline Reinforcement Learning for Profile Selection

Reinforcement learning (RL) maps observed state to actions that maximize expected long-term reward. In tabular Q-learning, an offline training update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta_{\text{RL}}\left[r_t + \gamma_{\text{RL}} \max_a Q(s_{t+1}, a) - Q(s_t, a_t)\right], \tag{2.14}$$

where η_{RL} is the learning rate and γ_{RL} is the discount factor [18, 19]. In this thesis, the action does not directly construct a route. It selects one of four fixed scoring profiles. This reduces the deployed decision to a finite mapping from a quantized state to a two-bit action.

Training and inference are separated. Training explores traces, updates Q-values, and freezes a selected policy. Deployment stores only the policy mapping needed for deterministic inference. The FPGA therefore contains no exploration mechanism and no runtime Q-value update path. A minimum-dwell mechanism can suppress rapid profile changes by holding the current action until a specified number of accepted decisions has elapsed.

This separation is particularly useful for a small deterministic accelerator. The training Q-table has $256 \times 4 = 1024$ signed Q8.7 entries and depends on exploration, episode sequencing, and reward accumulation. The deployed greedy policy requires only 256 two-bit actions. Removing online updates eliminates the learning-rate, discount, random-selection, and write-conflict hardware from the inference path. Policy changes remain possible, but they occur through an independently verified ROM image and bitstream rather than through unobserved online adaptation. ROM inference therefore provides bounded storage, a known one-cycle read

latency, and exhaustive state-by-state verification.

2.8 Fixed-Point Arithmetic and FPGA Implementation

Fixed-point arithmetic assigns an explicit integer and fractional width to every quantity. For a signed Q$m.n$ value, representable values and rounding rules depend on total width, signedness, scaling, saturation, and conversion policy. Bit-exact verification requires these details to be identical in the software reference and RTL. Saturation is preferable to silent wraparound for bounded metrics because it preserves the intended ordering at numeric limits.

FPGA logic can implement quantizers, ROMs, adders, comparators, and finite-state control with a deterministic clocked interface. Look-up tables (LUTs), registers, occupied slices, block memories, and timing reports characterize the physical mapping. Post-synthesis timing is an estimate before placement and routing; post-route timing includes the implemented netlist and routing delays and is the appropriate source for the maximum-frequency values used in the final comparison.

2.9 CPU, FPGA, and ASIC Characteristics

A CPU offers mature software tools, flexible control flow, and efficient handling of irregular workloads. It is appropriate for topology management, candidate generation, KMS/SDN orchestration, offline training, logging, and security administration. Its execution time can depend on operating-system scheduling, caches, and memory behavior unless a real-time environment is used.

An FPGA supports reconfigurable spatial datapaths and deterministic state machines. It is attractive for an evolving fixed-point kernel because profiles and interfaces can be revised by loading a new bitstream. A dedicated ASIC may ultimately provide greater density, performance, and energy efficiency, especially at high volume, but requires larger non-recurring engineering effort, signoff, fabrication, packaging, test, and redesign if the contract changes. Platform comparison must therefore be specific to development stage, volume, and measurement boundary.

2.10 Literature Review

2.10.1 QKD routing and network control

Bi et al. use virtual-network state and key resources for dynamic QKD routing [5]. Chen et al. study hybrid-trusted path selection [20]; Kiktenko et al. examine multiple non-overlapping paths [6]; and Akhtar et al. present fast and secure routing algorithms for QKD networks [21]. Zhang et al. jointly optimize routing, channels, key rate, and time slots [7]. These studies show

why hop count alone is insufficient, but their evaluated functions and evidence boundaries differ from a fixed-point candidate-evaluation kernel.

Recent work broadens this picture. Dynamic security-aware allocation couples route, wavelength, time slot, and security level [22]; trusted-node optimization minimizes activated relay sites subject to key-rate requirements [23]; and linear-programming formulations study KMS-level key forwarding [24]. Progressive recovery applies DRL to staged restoration after QKD-network failures [25], while moving-target defence changes routes under a zero-trust relay model [26]. These are complementary Class C studies: they justify resource, recovery, and policy awareness but neither implement nor time the same FPGA candidate kernel.

RL-based work includes deep-RL resource assignment [27], tabular Q-learning for resource-aware trusted-relay routing [8], and a graph-attention agent for flexible meshed topologies [9]. These studies provide broader network or generalization evidence than the present physical replay. They do not provide a same-input, same-function FPGA timing baseline.

Entanglement-network studies use RL or graph learning with different quantum-state and repeater models. Examples include qRL [28], deep-RL entanglement routing [29], and RELiQ's local-information graph approach [30]. They are relevant to adaptive routing as algorithmic context, but an entanglement fidelity in those models is not the same object as a post-processed classical-key pool or the bounded route-quality input in the present replay. No latency or quality ratio is transferred between these domains.

2.10.2 Operational QKD systems

MadQCI demonstrates heterogeneous SDN-QKD integration in production facilities [3]. Chen et al. report a carrier-grade quantum communication network extending beyond 10,000 km [4]. Such systems establish the practical setting for KMS and controller integration. Their scale and end-to-end objectives do not make their performance numbers directly comparable with a bounded FPGA kernel.

2.10.3 FPGA acceleration in QKD

Reconfigurable hardware has accelerated multidimensional reconciliation encoding, QKD post-processing, polar-code information reconciliation, error correction, and synchronization [10, 11, 12, 31, 13]. These contributions demonstrate that FPGA implementation is practical in QKD systems, but they process different functions and use throughput or latency boundaries that cannot be converted into route-selection speedups.

The wider hardware literature reinforces the same boundary. A reconciliation review surveys software and hardware implementation constraints [32]; FPGA-accelerated quantum-error-correction work frames real-time decoder requirements [33]; and simulator papers such as the QKD-platform survey and NetSquid address model construction rather than physical route-selection latency [34, 35]. RapidLayout uses evolutionary multi-objective search for an EDA

placement problem [36]; it is useful methodological context, not an FPGA-resident QKD routing baseline.

Hardware genetic-algorithm research provides relevant implementation ideas for pipelined evaluation, selection, and specialization [37, 38, 39, 40, 41]. It remains component-level context because those systems do not evaluate trusted-relay key-resource descriptors.

2.10.4 Comparison taxonomy

Table 2.3. Comparison classes used throughout the thesis.

Class	Definition	Valid comparison
A	Same function, board, interface, numeric contract, replay, and measurement boundary	Direct timing, latency, resource, exactness, and controller-behavior comparison
B	Same function and inputs on CPU/software and FPGA	Direct speed or energy comparison only when both are measured under a matched protocol
C	QKD routing and network-control algorithms	Blocking, utilization, quality, robustness, or generalization under compatible network experiments
D	Different QKD components, FPGA/RL hardware, deployments, or VLSI kernels	Architectural and feasibility context; no raw QFlow speed ratio

Class A is supplied internally by the five same-board configurations. No Class B experiment was completed. The QKD routing studies are Class C, while FPGA post-processing, operational deployments, hardware genetic algorithms, and isolated VLSI kernels are Class D. This taxonomy prevents a latency ratio between systems that perform different computations.

2.10.5 Reproducible literature search and verification

The literature record was verified on 17 July 2026 from a predefined 50-row review matrix. Each row was searched by exact title at an official publisher database or the official arXiv record, including IEEE Xplore, ACM Digital Library, SpringerLink, Nature, MDPI, Wiley/IET, Optica, Elsevier ScienceDirect, INFORMS, and Google Books where appropriate. The verification recorded bibliographic identity, platform, evidence layer, metric scope, and whether decision latency was actually reported. One review identifier was kept per matrix row; sources whose exact identity could not be verified were excluded, and explicit replacement sources were recorded as replacements rather than silently treated as the same publication.

The resulting disposition contained 21 Class C and 29 Class D records, with no identified Class A or Class B publication in the documented search. This is a bounded finding from the

recorded matrix and search date. Search queries, publisher locations, verification outcomes, and the final workbook are preserved under `literature/p1/` on the evidence branch identified in Appendix A.

2.10.6 Research position

Table 2.4. Representative related-work positioning. Direct numerical comparability requires an equivalent function and measurement boundary.

Work/category	Function	Evidence	Relationship to QFlow
Hao et al. [8]	Resource-aware adaptive routing	Network simulation	Class C RL counterpart; broader routing scope, no same-function FPGA
Johann et al. [9]	Generalizing DRL routing	Network simulation	Class C topology/generalization evidence; no direct kernel ratio
Bi et al. [5]	Dynamic key-resource routing	Software and network model	Class C resource-aware motivation
MadQCI [3]	SDN-QKD deployment	Operational testbed	Class D deployment context
Venkatachalam et al. [11]	QKD post-processing	FPGA implementation	Class D proof of FPGA suitability for a different function
This work	Fixed-point candidate evaluation	RTL, post-route, and physical replay	Class A internal same-board comparison with exact reference agreement

The research position is consequently specific. **QFlow** contributes physically validated fixed-point candidate-evaluation evidence, while several software studies contribute broader topology, algorithmic, or generalization evidence. Neither type of evidence dominates the other in every dimension.

2.11 Data-Analysis Principles

The evaluation uses exact integer comparison for reference/RTL/board agreement; post-route reports for maximum frequency and resources; accepted-request-to-done cycles for kernel latency; profile counts, transitions, and dwell lengths for adaptive behavior; and paired confidence intervals, sign tests, and effect sizes for held-out route-quality comparisons. Missing power, energy, or CPU results are treated as unmeasured rather than as zero.

2.12 Conclusion

The background motivates key-resource-aware routing, separates upstream eligibility from profile-conditioned FPGA preference, and explains deterministic policy-ROM inference. The literature taxonomy positions **QFlow** without invalid cross-function speed comparisons.

CHAPTER 3
METHODOLOGY

3.1 Introduction

This chapter defines the final network-control model and, separately, the exact contract implemented and replayed on the FPGA. It then specifies candidate fields, fixed-point scoring, configurations H0–H4, controller thresholds, the H3 rule, the H4 policy and reward, training partitions, RTL structure, cycle measurement, physical transport, verification, and statistics. The separation is essential: the network model describes deployable classical key management, whereas the completed H4 experiment establishes deterministic controller and hardware behavior under its registered synthetic inputs.

3.2 Two Related but Distinct Contracts

The thesis uses two contracts that answer different questions.

1. The *network-control model* represents a trusted-relay QKD network after link post-processing. It uses classical key-pool occupancy, admitted key generation, key consumption, link status, utilization, policy freshness or revocation, and externally measured health indicators. It is the intended integration model for a KMS/SDN controller.
2. The *completed H4 experiment* uses the four registered controller features: minimum key occupancy, bottleneck route-quality code, offered request load, and utilization imbalance. Its second feature retains the registered "fidelity" field name in source artifacts, but its valid interpretation here is a bounded experimental route-quality code.

The second feature of the completed H4 implementation is therefore treated as an abstract controller input. It is not interpreted as quantum coherence, as fidelity decay of post-processed keys in classical storage, or as a measured replenishment rate. Results from that controller support policy-ROM inference, fixed-point exactness, latency, resource, and trace-specific adaptive behavior. They are not evidence that the broader network model has already been validated with live QKD equipment.

3.3 System Overview and Boundary

At a decision epoch, an upstream trusted controller supplies a bounded candidate set and four state features. The FPGA samples one request, selects a profile, evaluates the supplied candidates, and returns a registered result. Candidate generation, graph traversal, authentication, KMS access, security authorization, and route installation remain outside the kernel.

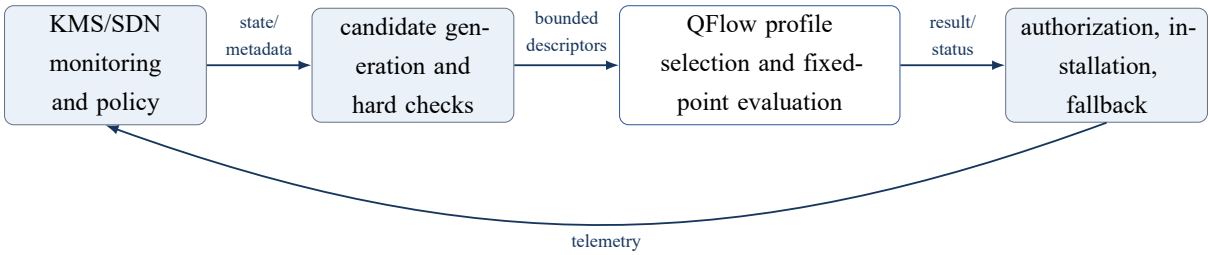


Figure 3.1. System boundary. Only profile selection and candidate evaluation are included in the measured FPGA kernel.

The kernel-cycle boundary begins on a synchronous edge satisfying `start && ready` and ends on the synchronous `done` edge. UART serialization, host parsing, operating-system scheduling, USB/JTAG transfer, programming time, candidate generation, and route installation are excluded.

3.4 Final Network and Key-Resource Model

Let $G = (V, E)$ be a directed trusted-relay QKD network. For link (i, j) at time t, the KMS maintains classical key units $K_{ij}(t)$ with configured capacity $K_{ij}^{\max}$. A discrete update is

$$K_{ij}[n+1] = \mathrm{clip}_{[0,K_{ij}^{\max}]}\big(K_{ij}[n] + G_{ij}[n] - C_{ij}[n] - X_{ij}[n]\big), \tag{3.1}$$

where G_{ij} is newly admitted secret material, C_{ij} is consumption, and X_{ij} is expiry, revocation, or administrative removal. The normalized occupancy is $o_{ij} = K_{ij}/K_{ij}^{\max}$.

For a request demanding D key units, a link-level hard eligibility indicator is

$$\chi_{ij}(D) = \mathbf{1}\{\mathrm{status}_{ij} = \mathrm{UP}\}\mathbf{1}\{K_{ij} \geq D\}\mathbf{1}\{q_{ij} \leq q_{\mathrm{abort}}\}\mathbf{1}\{\mathcal{M}_{ij}\ \mathrm{valid}\}, \tag{3.2}$$

where q_{ij} is an externally supplied QBER or health measurement and $\mathcal{M}_{ij}$ contains authenticated policy metadata, including freshness or revocation state. Route $\mathcal{R}_i$ is eligible when

$$v_i = \prod_{(u,v)\in\mathcal{R}_i} \chi_{uv}(D) = 1. \tag{3.3}$$

These hard conditions are not replaced by a weighted score. The upstream KMS/SDN controller evaluates Equations (3.2) and (3.3), then supplies one validity bit v_i with each bounded candidate descriptor. The physical FPGA does not recompute link status, key sufficiency, QBER, or policy metadata; it gates candidates using the supplied bit, excluding invalid slots from range formation and winner selection. Freshness is a classical KMS policy property; the model contains no stored-key coherence-decay term.

3.5 Executed Request and Candidate Contract

Table 3.1. Executed candidate fields used by H0–H4; aggregation precedes fixed-width sampling. Slots 0–1 carry Ring-6 candidates, slots 2–3 are invalid, and zero feasible candidates produces `NO_PATH`.

Field	Symbol	Meaning/source	Route aggregation	Width	Format	Range	Saturation or exceptional rule
Candidate valid	v_i	Upstream eligibility result	Logical AND of link and request checks	1	unsigned Boolean	0–1	Invalid candidates never enter min/max or winner selection
Path cost	c_i	Precomputed candidate cost	Sum defined by the software environment or supplied replay	32	unsigned UQ16.16	$0–2^{16}-2^{-15}$	Maximum finite code is `0xFFFFFFFE`; `0xFFFFFFFF` is infinity
Bottleneck route quality	q_i	Registered bounded route-quality input	Minimum code along the route in the software experiment	16	unsigned UNORM16	0–65,535, representing 0–1	Clip to $[0, 1]$, multiply by 65,535, round half up
Utilization objective	u_i	Candidate utilization/imbalance input	Maximum candidate-link utilization in the software experiment	32	unsigned UQ16.16	$0–2^{16}-2^{-16}$	Full unsigned range through `0xFFFFFFFF`; no reserved infinity code
Hop count	h_i	Number of directed edges	Count of edges in route	3	unsigned integer	0–7	Values outside the interface width are invalid upstream
Candidate slot	r_i	Stable candidate identifier	Assigned by generator	2	unsigned integer	0–3	Used as final deterministic tie-break

Table 3.2. Executed controller-state input fields.

Field	Width/format	Registered meaning	Saturation
Minimum key occupancy x_K	16, UNORM16	Minimum $K/16$ over links appearing in either supplied Ring-6 candidate	Clip to $[0, 1]$
Bottleneck route quality x_Q	16, UNORM16	Minimum bounded route-quality input over those links; artifact field name is fidelity	Clip to $[0, 1]$
Offered request load x_L	16, UNORM16	Offered requests in the control window divided by capacity four	Clip to $[0, 1]$
Utilization imbalance x_I	16, UNORM16	Maximum minus minimum link utilization over candidate links; 16-step consumption window	Clip to $[0, 1]$

All four state ports and all UNORM16 candidate codes use the same conversion:

$$\mathrm{UNORM16}(x) = \mathrm{clip}_{[0,65535]} \left\lfloor 65535\, \mathrm{clip}_{[0,1]}(x) + \tfrac{1}{2} \right\rfloor . \tag{3.4}$$

Thus one least-significant unit represents $1/65535 \approx 1.5259 \times 10^{-5}$ of full scale.

3.6 Fixed-Point Objective Formation and Selection

For each feasible candidate, the three minimization objectives are

$$f_{i,c} = c_i, \qquad f_{i,q} = 65535 - q_i, \qquad f_{i,u} = u_i. \tag{3.5}$$

The route-quality deficit converts a larger-is-better code into a smaller-is-better objective. For objective j, define the ideal point and observed range over feasible candidates as

$$z_j^\star = \min_{i:v_i=1} f_{i,j}, \qquad F_j^{\max} = \max_{i:v_i=1} f_{i,j}. \tag{3.6}$$

The ideal point is a componentwise reference and can combine values from different routes. The normalized penalty is

$$N_{i,j} = \begin{cases} 0, & F_j^{\max} = z_j^\star, \\ \mathrm{round}_{\text{half up}} \left(\dfrac{(f_{i,j} - z_j^\star)65535}{(F_j^{\max} - z_j^\star) + 1} \right), & \text{otherwise.} \end{cases} \tag{3.7}$$

The $+1$ in the denominator keeps the result finite and inside UNORM16. A single shared iterative 49-by-33-bit unsigned divider performs the nontrivial range normalizations.

For profile Π_k, the 18-bit score is

$$S_i^{(k)} = \max\{\rho_c^{(k)} N_{i,c}, \rho_q^{(k)} N_{i,q}, \rho_u^{(k)} N_{i,u}\}, \tag{3.8}$$

where each ρ is a two-bit unsigned ratio. Lower score is better. The complete ranking key is

$$\mathcal{K}_i = (S_i^{(k)}, c_i, h_i, r_i), \tag{3.9}$$

compared lexicographically in ascending order. Therefore score ties fall back to raw path cost, then hop count, then candidate slot. If $N_c = 0$, score and returned quality are zero, the selected identifier is 255, and status is `NO_PATH`.

3.6.1 Why both alpha and Tchebycheff weights exist

For training, action Π_k constructs path cost from link-level variables using the full four-coefficient vector

$$c_i^{(k)} = \sum_{(u,v)\in\mathcal{R}_i} \left(\frac{\alpha_1^{(k)}}{K_{uv}} + \frac{\alpha_2^{(k)}}{Q_{uv}} + \frac{\alpha_3^{(k)}}{R_{uv}} + \alpha_4^{(k)} B_{uv} \right), \tag{3.10}$$

where K_{uv} is an integer available-key count, Q_{uv} is a dimensionless route-quality value in $[0, 1]$, R_{uv} is the positive synthetic key-rate value, and B_{uv} is dimensionless synthetic QBER in $[0, 1]$. The training generator uses $R_{uv} \in [3, 9]$ as a contract value rather than a calibrated physical-rate measurement. Hence each reciprocal term penalizes scarcity: lower key count, quality, or synthetic rate increases cost, while higher QBER increases it directly. Before Equation (3.10) is evaluated, software rejects a path containing $K_{uv} < 1$, $Q_{uv} < 0.90$, or $R_{uv} \leq 0$, preventing division by zero. The fixed-point export likewise maps a zero denominator to `0xFFFFFFFF` (infinity), while a finite overflow saturates at `0xFFFFFFFE`.

Equation (3.10) produces one path-cost objective. During training, the same action Π_k then uses all three Tchebycheff weights to trade that cost against route-quality deficit and utilization imbalance. Thus the four α coefficients and three objective ratios both affect a software-training action. In the physical replay, by contrast, c_i is precomputed, supplied to the FPGA, and common across profiles. The 18-bit ROM word still contains the α fields, but the physical candidate evaluator reads only the lower six ratio bits. Physical exactness therefore validates the ratio-only deployment projection, not numerical equivalence to the full training action or on-FPGA evaluation of Equation (3.10). The full training action, frozen artifacts, and ratio-only physical projection are distinguished explicitly in Figure 3.2.

3.7 Registered Profile Codebook

Table 3.3. Profile semantics and exact registered coefficients. "Low-latency" means path-cost/hop preference; it does not mean shorter FPGA evaluation time.

ID	Registered name	$\boldsymbol{\alpha}^{(k)}$	$\boldsymbol{\lambda}_{\mathrm{TCH}}^{(k)}$	HW ratio	Payload
Π_0	Balanced	$(1.0, 1.5, 0.5, 2.0)$	$(.40, .40, .20)$	$(2, 2, 1)$	`0x13329`
Π_1	Scarcity protection	$(1.5, 1.5, 1.0, 2.0)$	$(.25, .25, .50)$	$(1, 1, 2)$	`0x1B516`
Π_2	Fidelity protection	$(1.0, 2.0, 0.5, 3.0)$	$(.25, .50, .25)$	$(1, 2, 1)$	`0x14399`
Π_3	Low-latency	$(0.5, 1.0, 0.5, 1.0)$	$(.50, .25, .25)$	$(2, 1, 1)$	`0x0A2A5`

The registered names are retained, but their deployed numerical meanings are exact: Π_1 gives the utilization/scarcity-management objective the strongest ratio, Π_2 gives route-quality deficit the strongest ratio, and Π_3 gives supplied path cost the strongest ratio. "Fidelity protection" refers to the bounded route-quality code, not coherence decay in a classical key store. Each α is packed as a three-bit half-unit numerator, followed by three two-bit Tchebycheff ratios. Multiplying a ratio vector by a common constant does not alter the selected route, so normalization of $(2, 2, 1)$ to $(0.4, 0.4, 0.2)$ is unnecessary for the integer argmin. The α fields affect training through Equation (3.10); only the ratio fields affect the deployed FPGA selection described here.

3.8 Exact Definitions of Configurations H0–H4

All five configurations use the same XC6SLX16 target, board shell, clock, candidate widths, result schema, replay, handshake, and constraint file. They are synthesized, placed, and routed independently.

Table 3.4. Exact selection function of the five physical configurations.

Mode	Controller	Winner rule	Purpose
H0	Feasible minimum-hop baseline	$\arg\min_{v_i=1}(h_i, r_i)$	Valid candidate with smallest hop count; candidate-slot tie-break
H1	Fixed key-aware cost-only	$\arg\min_{v_i=1}(N_{i,c}, c_i, h_i, r_i)$; ratio $(1, 0, 0)$	Cost-only evaluator baseline; no profile-ROM read
H2	Fixed QFlow profile	Equation (3.8) with Π_0 ratio $(2, 2, 1)$	Full fixed-point candidate datapath
H3	Threshold/rule adaptive	Rule in Table 3.6, followed by the selected ratio	Deterministic adaptive-controller baseline
H4	Reinforcement-learned adaptive	256-by-2 policy ROM, three-decision dwell, then selected ratio	Offline-learned controller under deterministic inference

H0 is deliberately not function-equivalent to H1–H4. H1 evaluates only cost and is not a full-profile comparator. H2 isolates the complete scoring datapath under a constant profile. H3 and H4 share the profile-conditioned datapath, making H4 versus H3 the fairest adaptive-controller comparison.

Pseudocode 3.1. Common candidate reduction

Input: offered candidate slots, selected profile ratio $\boldsymbol{\rho}$.
Output: selected slot, score, quality code, and status.

1. Validate the request framing, accept the upstream-supplied validity bits, and form $\mathcal{F} = \{i : v_i = 1\}$.
2. If the request is malformed, return `INVALID_REQUEST`.
3. If $\mathcal{F} = \varnothing$, return `NO_PATH`.
4. Form cost, quality-deficit, and utilization ranges over $\mathcal{F}$.
5. Normalize each objective using Equation (3.7).
6. Compute the applicable H0, H1, or profile score.
7. Retain the smallest ranking key from Equation (3.9).
8. Register the winner and assert `done` for one cycle.

3.9 Controller Quantization and State Encoding

Each feature is quantized into four levels. For thresholds $\theta_1 < \theta_2 < \theta_3$,

$$b(x) = \begin{cases} 0, & x < \theta_1, \\ 1, & \theta_1 \le x < \theta_2, \\ 2, & \theta_2 \le x < \theta_3, \\ 3, & x \ge \theta_3. \end{cases} \tag{3.11}$$

Equality always enters the higher bin. Table 3.5 gives both real and encoded thresholds; these are the values compared in the RTL.

Table 3.5. Actual controller thresholds and UNORM16 encodings.

Feature	θ_1	θ_2	θ_3	**Encoded thresholds**	**Source/meaning**
Minimum key occupancy	0.25	0.50	0.75	16,384; 32,768; 49,151	Registered capacity-normalized pool thresholds
Bottleneck route quality	0.90	0.93	0.96	58,982; 60,948; 62,914	Registered experimental route-quality thresholds
Offered request load	0.25	0.50	0.75	16,384; 32,768; 49,151	Requests divided by window capacity four
Utilization imbalance	0.125	0.25	0.50	8,192; 16,384; 32,768	Registered max-minus-min utilization thresholds

Let the four bins be b_K, b_Q, b_L, b_I. The radix-4 address is

$$z = (((b_K \times 4) + b_Q) \times 4 + b_L) \times 4 + b_I, \qquad z \in \{0, \ldots, 255\}. \tag{3.12}$$

In RTL, $z[7{:}6] = b_K$, $z[5{:}4] = b_Q$, $z[3{:}2] = b_L$, and $z[1{:}0] = b_I$. This ordering is used by both the H3 rule and H4 policy ROM.

3.10 H3 Rule Controller

H3 applies the first matching row in Table 3.6. The table is an executable priority specification, not an informal description.

Table 3.6. Exact first-match H3 rule mapping.

Priority	Condition	Profile	Numerical effect in the physical evaluator
1	$b_K = 0$	Π_1	Ratio $(1, 1, 2)$
2	Otherwise, $b_Q \leq 1$	Π_2	Ratio $(1, 2, 1)$
3	Otherwise, $b_I \geq 2$	Π_1	Ratio $(1, 1, 2)$
4	Otherwise, $b_L = 3 \wedge b_K \geq 2 \wedge b_Q \geq 2 \wedge b_I \leq 1$	Π_3	Ratio $(2, 1, 1)$
5	Otherwise	Π_0	Ratio $(2, 2, 1)$

For example, a state with both critical occupancy and low route-quality satisfies rows 1 and 2, but row 1 selects Π_1. This priority is part of the baseline and prevents an implementation from reordering conditions for convenience.

3.11 H4 Policy-ROM and Dwell Inference

The H4 policy ROM contains 256 state-major two-bit actions and has one registered read cycle. The action population is 145 entries selecting Π_0, 43 selecting Π_1, 44 selecting Π_2, and 24 selecting Π_3. The profile ROM contains four 18-bit payloads. The training Q-table contains 1,024 signed 16-bit Q8.7 values and is retained for analysis only; it is absent from the inference datapath.

The minimum-dwell controller stores the active profile and the number of valid accepted decisions since the last change. A requested change is admitted only after the registered dwell of three decisions has been satisfied. A no-path result is a valid controller decision and advances policy/dwell state; a malformed request is rejected as invalid and does not become an adaptive decision. Reset clears profile history and the dwell counter.

Pseudocode 3.2. H4 inference

Input: four UNORM16 state features and current dwell state.
Output: deployed profile $a[n]$.

1. Quantize x_K, x_Q, x_L, x_I with Table 3.5.
2. Form address $z[n]$ using Equation (3.12).
3. Read requested action $a_{\text{req}} = \pi(z[n])$ from the policy ROM.
4. If this is the initial decision, initialize the active action deterministically.
5. If a_{req} differs from the active action and the dwell requirement is not satisfied, retain the active action.

6. Otherwise admit the requested action and reset the dwell counter on change.
7. Read the 18-bit profile word and evaluate the candidate set.

There is no runtime argmax over Q-values, no exploration, no Q update, and no online learning on the FPGA. "Reinforcement-learned" refers to the offline origin of the ROM mapping; the corresponding deployment branch is summarized in Figure 3.2.

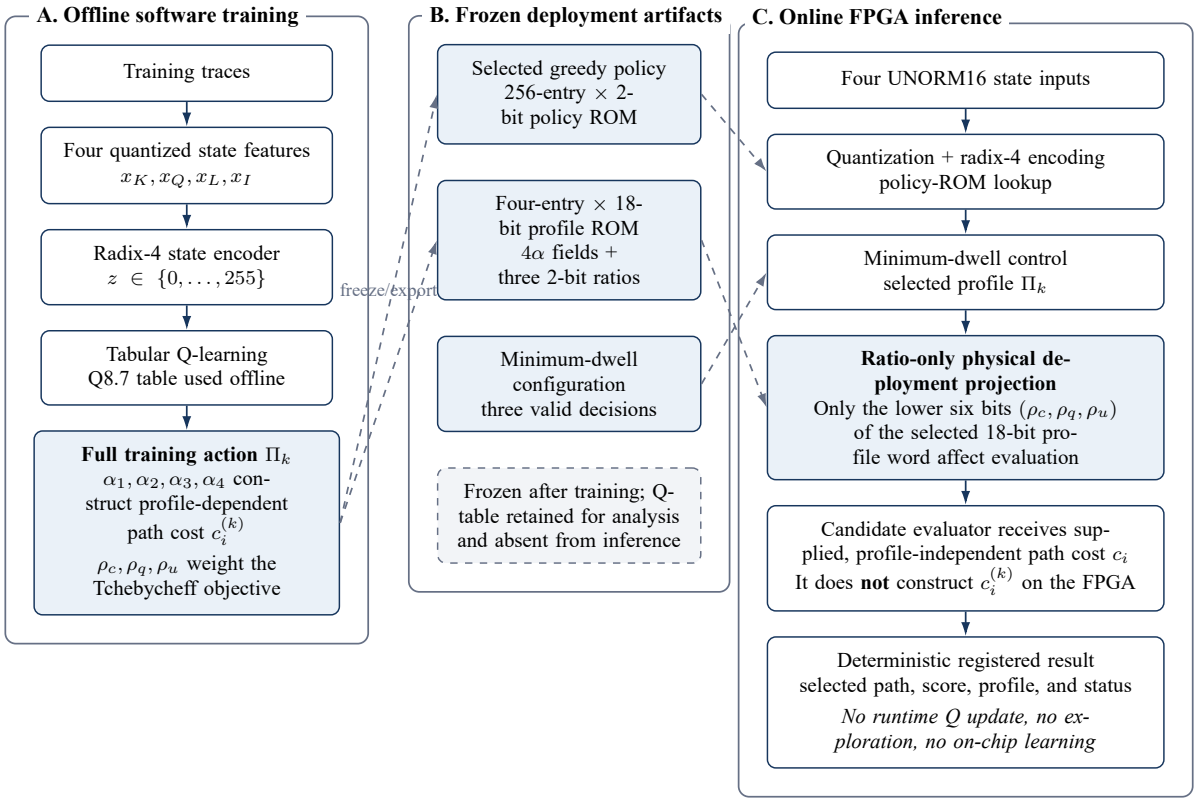


Figure 3.2. Offline training and FPGA deployment path. Software training uses the complete profile action: coefficients α_1–α_4 construct profile-dependent path cost and ratios (ρ_c, ρ_q, ρ_u) weight the Tchebycheff objective. The physical implementation deploys the frozen state-to-profile policy, minimum-dwell control, and only the objective-ratio projection over externally supplied profile-independent path cost; it performs no runtime Q update, exploration, on-chip learning, or on-FPGA construction of $c_i^{(k)}$.

3.12 Immediate Reward and Q-Learning Update

The original evaluation reward is defined for each decision by success indicator y_t, selected bottleneck route-quality value $b_t \in [0, 1]$, post-decision utilization imbalance I_t^{post}, selected hop

count h_t, action a_t, and previous action a_{t-1}. Define

$$U_Q(b_t) = \text{clip}_{[0,1]}\left(\frac{b_t - 0.90}{1 - 0.90}\right), \tag{3.13}$$

$$U_B(I_t^{\text{post}}) = 1 - \text{clip}_{[0,1]}(I_t^{\text{post}}), \tag{3.14}$$

$$\sigma_t = \mathbf{1}\{a_{t-1} \text{ exists and } a_t \neq a_{t-1}\}. \tag{3.15}$$

For a successful decision,

$$r_t^{\text{eval}} = 4 + 2U_Q(b_t) + U_B(I_t^{\text{post}}) - 0.25h_t - 0.25\sigma_t. \tag{3.16}$$

For a blocked decision,

$$r_t^{\text{eval}} = -4 - 0.25\sigma_t, \tag{3.17}$$

and the quality, balance-utility, and hop components are zero. The declared nominal evaluation range is $[-4.25, 6.75]$.

The executed training target changes only the successful-decision balance coefficient:

$$r_t^{\text{train}} = 4 + 2U_Q(b_t) + 4U_B(I_t^{\text{post}}) - 0.25h_t - 0.25\sigma_t. \tag{3.18}$$

For a blocked training decision, $U_B = 0$, so the executed rule remains

$$r_t^{\text{train}} = -4 - 0.25\sigma_t. \tag{3.19}$$

Table 3.7. Reward components used by the registered learner and final evaluation.

Component	Coefficient	Direction and range	Interpretation/limitation
Success	$+4$	Positive, once per selected route	Dominant service outcome
Blocking	-4	Negative, when no route is selected	Does not distinguish the physical causes of infeasibility
Route quality	$+2$	0–2 after clipping above 0.90	Abstract experimental route-quality utility, not stored-key coherence
Balance utility	$+1$ in r_t^{eval}; $+4$ in r_t^{train}	Larger when post-decision imbalance is smaller	Training increases balance emphasis; reported evaluation uses the common $+1$ scale
Hop cost	-0.25 per edge	More hops reduce reward	A topological proxy, not measured end-to-end delay
Profile switch	-0.25	Applied after the first action when the action changes	Discourages control chattering; previous action is controller history

The reward r_t^{train} in Equations (3.18)–(3.19) shapes the learned Q-values and accompanies the three-decision dwell mechanism. Validation, held-out testing, Table 4.1, and the final reported metrics use r_t^{eval} from Equations (3.16)–(3.17), preserving a common scale for comparing completed controllers.

Tabular Q-learning uses

$$Q(z_t, a_t) \leftarrow Q(z_t, a_t) + 0.1 \left[r_t^{\text{train}} + 0.95 \max_a Q(z_{t+1}, a) - Q(z_t, a_t) \right], \tag{3.20}$$

with the terminal bootstrap omitted after the last record. The reported value 4.6771 is a mean of r_t^{eval}, not a training-target mean. On that evaluation scale, a mean near 4.67 means that successful decisions dominate and usually earn positive normalized route-quality and balance utility, partly offset by hop and switch costs. It is a dimensionless score under this particular reward definition, not money, energy, secret-key rate, or a universal network utility.

3.13 Training Environment and Trace Generation

3.13.1 Topology, candidates, and transition order

The controlled software environment is a directed Ring-6 with 12 directed edges. For each source–destination request, the generator supplies the clockwise and counter-clockwise simple paths. These occupy candidate slots 0 and 1; the four-slot hardware contract reserves slots 2

and 3. On a successful step, one key unit is consumed from every link of the selected route. The offered-request count is a state/load variable in $\{0, 1, 2, 3, 4\}$; it is not a claim that a single transition consumes that many units.

The software step order is:

1. observe the current four-feature state;
2. choose one of four profile actions;
3. use all four action-specific α coefficients to construct the two path costs, then use all three action-specific ratios to select a route or block;
4. consume one key unit per selected link on success;
5. apply the trace's integer key arrivals and registered quality/rate/QBER update;
6. advance the trace index; and
7. observe the next state unless the final record was consumed.

No-path is nonterminal. An episode terminates only after record 512. The utilization feature uses consumption over the preceding 16 steps divided by the current key count, with denominator at least one.

3.13.2 Initialization and phase construction

Every generated trace begins with 12 independently generated directed-link states. Initial key count is an integer from 6 through 12 inclusive; initial route-quality input is 0.9400–0.9900; the synthetic key-rate value is 3–9; and QBER is 0.0150–0.0550. Link capacity is 16 and request-window capacity is four.

Trace phases change every 32 steps and cycle through nominal, scarcity, route-quality stress (registered as fidelity), high load, low latency, and mixed. High-load phases set offered load to four; other phases draw an integer from zero through four using the trace generator's Xor-Shift32 stream. The low-latency phase changes the source–destination pair from $(0, 3)$ to $(0, 2)$; other phases use $(0, 3)$. Phase-specific link updates create deterministic differences between the upper and lower ring directions. These labels describe synthetic trace regimes and must not be interpreted as observations from deployed quantum links. Figure 3.3 relates the two occupied candidate slots to the directed topology and shows the registered phase and post-decision update sequence.

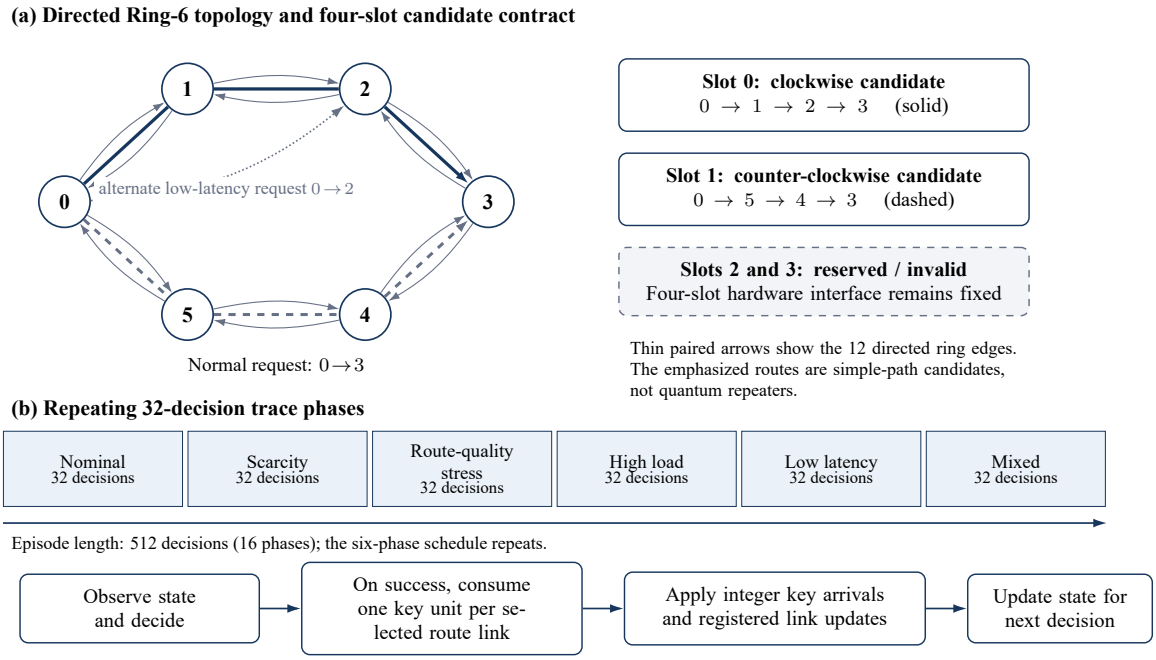


Figure 3.3. Controlled Ring-6 training and evaluation environment. Thin paired arrows show the 12 directed ring edges; solid slot 0 and dashed slot 1 carry the clockwise and counter-clockwise simple paths for the normal $0 \to 3$ request, while slots 2 and 3 remain invalid in the four-slot hardware contract. The 512-decision traces use repeating 32-decision operating phases, with link consumption followed by integer arrivals, registered link updates, and the next state after each decision.

Table 3.8. Registered trace ranges and their methodological role.

Quantity	**Registered range/rule**	**Role**
Initial key count	Integer 6–12; capacity 16	State occupancy and route feasibility
Initial route-quality input	0.9400–0.9900	Abstract bounded controller/path metric
Initial synthetic key rate	3–9	Software path-cost and feasibility input
Initial QBER	0.0150–0.0550	Software path-cost input
Offered request load	Integer 0–4; high-load phase fixed at 4	State feature after division by four
Episode length	512 decisions	Common finite training/evaluation horizon
Phase length	32 decisions	Repeatable nonstationary regime schedule

In the software environment, a candidate is omitted when any path link has fewer than one key unit, a route-quality value below 0.90, or nonpositive synthetic key rate. If neither direction remains, the step blocks but the trace still advances. The separate physical replay also contains explicitly malformed records to test `INVALID_REQUEST`; those records are protocol tests, not samples from the training environment.

3.13.3 Partitions, seeds, and independence

Table 3.9. Exact trace and trainer seeds. No generated trace appears in more than one partition.

Use	Seeds
Training traces	101, 211, 307, 401, 503, 601, 701, 809
Validation traces	1009, 1103, 1201, 1301
Test traces	2003, 2111, 2203, 2309
Trainer RNG	17, 29, 43, 71, 113, 167, 229, 283

Distinct seeds make the generated partitions disjoint and the trainer runs repeatable, but they do not make all time steps statistically independent: records within a trace share topology, evolving pools, and phase schedule. Statistical summaries therefore use seeds or paired physical rows as their declared sampling units rather than treating every transition as an independent network realization.

3.13.4 Training schedule, ranking, and policy freeze

The Q-table starts at zero. Each trainer candidate runs 200 fixed epochs, eight episodes per epoch, and 819,200 transitions. Epsilon decreases linearly from 1.0 to 0.05 over the first 80% of transitions and remains 0.05 thereafter. Evaluation uses $\epsilon = 0$. Each epoch shuffles the eight training traces by XorShift32 and Fisher–Yates. During exploratory training, equal maximum Q-values are broken uniformly with XorShift32; validation, testing, and deployment choose the smallest action identifier on equality.

There is no early stopping or hyperparameter sweep. A candidate must use at least two profiles, allocate at least 1% to the second-most-used profile, remain within an absolute blocking margin of 0.005, and keep switch rate at or below 0.25. The test partition was inaccessible until policy selection was complete. Among seven eligible trainer seeds, seed 229 was selected mechanically by highest validation reward, followed in order by lower blocking, higher route quality, lower switching, and smaller seed. The resulting policy and profile images were then used without online modification.

3.14 Detailed RTL Architecture

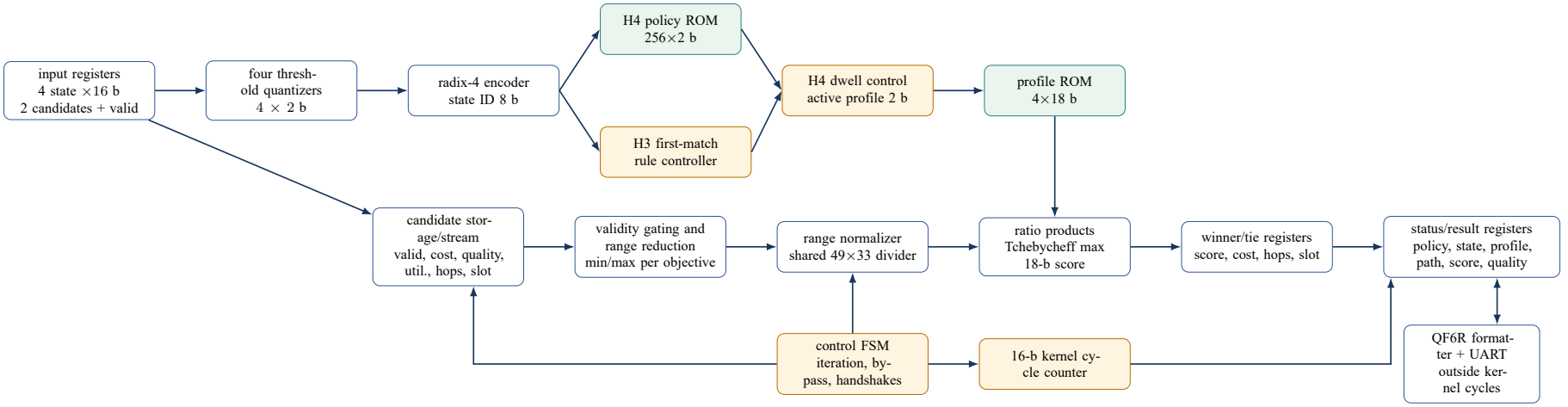


Figure 3.4. Detailed QFlow RTL architecture. H0 bypasses the profile and iterative score path; H1 bypasses profile-ROM lookup; H2 fixes Π_0; H3 uses the rule controller; H4 uses policy ROM and dwell control. Formatter and UART time are not included in the kernel-cycle measurement.

Input registers decouple request sampling from multi-cycle evaluation. State quantizers and the radix-4 encoder are shared by adaptive modes. The configuration parameter selects the H0, H1, H2, H3, or H4 controller path at synthesis time. The candidate engine collects objective extrema, bypasses division when a range is zero, otherwise iterates the shared divider, forms three weighted penalties, and updates the winner registers. The FSM sequences all stages and emits exactly one done pulse. The cycle counter wraps the unchanged policy shell and observes the accepted-edge to done-edge distance.

3.15 Cycle-Level Operation

A start is accepted on edge N_{accept} only when `ready` is high. The wrapper zeroes its counter on that edge. Initialization clears range and winner registers; candidate collection records feasibility and extrema; the evaluator then normalizes nonconstant objective ranges, computes the score, compares the winner, and registers the result. On the one-cycle done pulse,

$$L = N_{\text{done}} - N_{\text{accept}}, \qquad t = L(10\ \text{ns}) \tag{3.21}$$

at the physical 100 MHz clock.

Table 3.10. Observed normal-result cycle structure. The min/max/median values are measured from the common replay, not estimates from a nominal pipeline formula.

Mode	Min	Mean	Median	Max	Engineering explanation
H0	2	2.000	2	2	Direct feasible minimum-hop comparison followed by registered completion; iterative evaluator bypassed
H1	7	201.763	309	309	Cost-only range normalization; profile-ROM read bypassed
H2	8	202.763	310	310	H1-style evaluation plus one profile-ROM setup cycle for fixed Π_0
H3	8	202.763	310	310	Rule selection completes within the same setup envelope as H2
H4	10	204.763	312	312	Policy-ROM and dwell/controller pipeline add two cycles relative to H3

The shared iterative divider dominates H1–H4. Equal objective ranges and single-feasible-candidate cases bypass some iterations, which explains the small minimum but much larger median and the noninteger mean over 76 normal rows. H0 is roughly two cycles because it never enters this normalization path. H1 is one cycle shorter than H2/H3 because its cost-only controller does not read the profile ROM. H4 adds exactly two setup cycles to the otherwise

comparable H3 datapath. Thus H4 versus H3 is the appropriate controller-overhead comparison; H4 versus H0 mixes a controller change with the addition of the complete multi-objective evaluator.

3.16 Physical Result Protocol

The synchronous shell returns the nine fields in Table 3.11. Every one was compared with the software reference for every physical record.

Table 3.11. Nine exact fields in the physical replay result.

Field	Decimal width	Range	Meaning
`test_id`	4	0–9999	Stable replay record identifier
`policy_id`	1	0–4	Physical configuration H0–H4
`state_id`	3	0–255	Encoded controller state; zero for invalid request
`profile_id`	1	0–3	Applied profile; zero for invalid request
`selected_path`	3	0–3 or 255	Candidate slot, or 255 on failure
`score`	6	0–262143	Unsigned 18-bit winner score; zero on failure
`bottleneck_fidelity`	5	0–65535	Registered name for returned route-quality code; zero on failure
`cycles`	3	0–999	$N_{\text{done}} - N_{\text{accept}}$
`status`	1	0–2	Completion status

Status 0 is `OK`, status 1 is `NO_PATH`, and status 2 is `INVALID_REQUEST`. For status 1 or 2 the selected path is 255 and score and returned quality are zero. An accepted invalid request completes in one kernel cycle. A start presented while `busy` is high is not accepted, does not reset the counter, produces no separate response, and cannot disturb the active request.

The transport record is exactly

```
QF6R,1,TTTT,P,SSS,R,PPP,CCCCCC,FFFFF,LLL,E\r\n
```

and is 44 ASCII bytes including CRLF. Fields are unsigned decimal with intentional leading zeroes and are emitted left to right in the order shown. The production link is 115,200 baud, 8 data bits, no parity, one stop bit, idle high, and no flow control. Each UART frame sends a low start bit, data bits least-significant-bit first, and a high stop bit. The rounded divider is 868 clocks per serial bit. The formatter uses 18 iterative shift-add-3 steps per numeric field; formatter and UART latency are outside Equation (3.21).

3.17 Verification Methodology

Verification proceeds from pure functions to physical records. A higher level is entered only after the lower-level numeric and interface contracts pass.

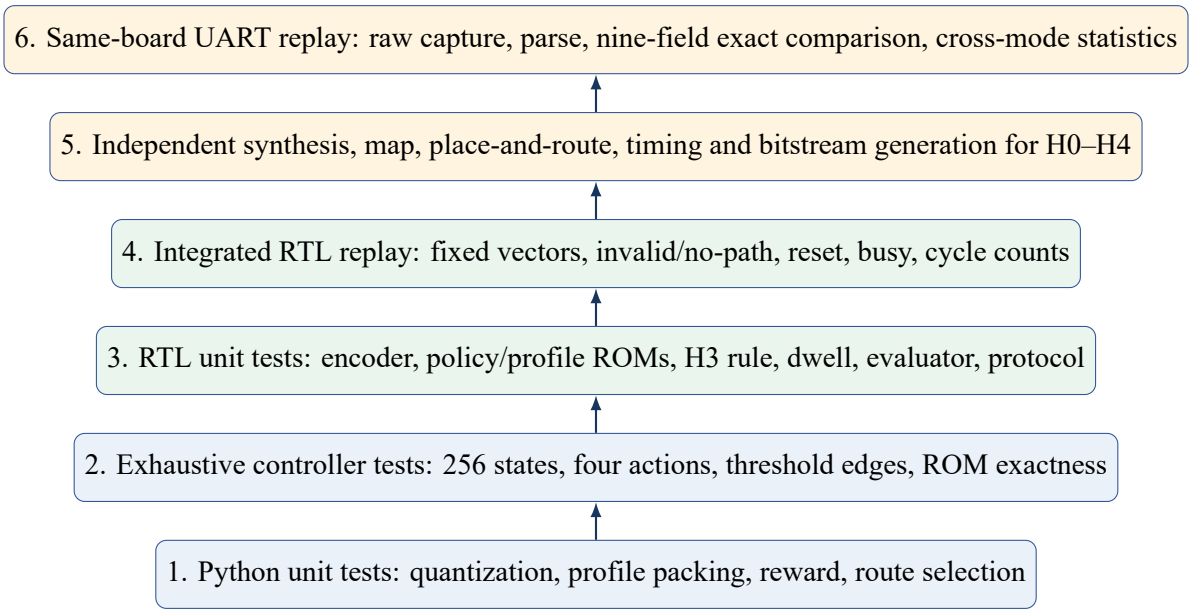


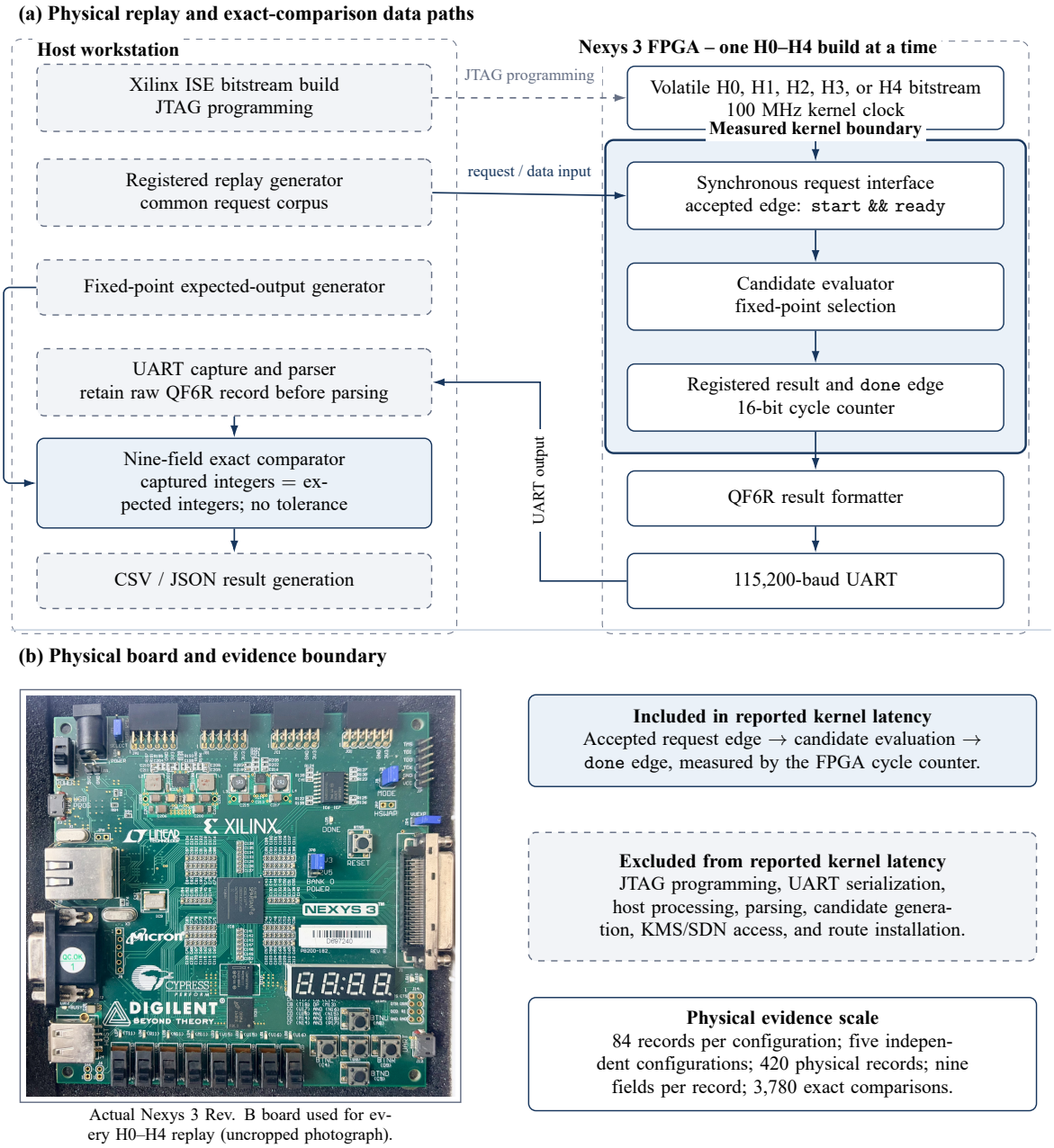
Figure 3.5. Verification ladder from executable arithmetic to same-board physical capture. Each level tests a distinct failure class.

The software suite includes 74 regression tests. Exhaustive tests cover all 256 state addresses, all four actions, all policy and profile ROM words, and below/at/ above cases for every quantizer threshold. RTL tests also cover both candidates, ties, equal objective ranges, one feasible candidate, no path, invalid request, reset, back-to-back acceptance, and a start while busy. Integrated tracing retained 2,125 cycle vectors and 530 valid decisions. Physical, post-route, RTL, and software evidence are reported separately.

3.18 FPGA Implementation and Physical Replay

The target is a Digilent Nexys 3 Rev. B containing a Xilinx XC6SLX16-2-CSG324 Spartan-6 FPGA. Xilinx ISE 14.7 independently builds H0–H4. The physical clock is 100 MHz; post-route $f_{\max}$ is implementation capacity and is not substituted for the board clock when converting cycles to time.

The common corpus contains 20 contract rows and 64 held-out dynamic rows, for 84 records per configuration. For each build, the host programs the volatile bitstream, transmits one registered request at a time, stores the raw UART capture, parses the nine output fields, and compares them with the fixed-point reference. The experiment therefore contains 420 physical records and 3,780 field comparisons. The held-out partition was materialized with seed 26072026 before the cross-mode result tables were generated.



Actual Nexys 3 Rev. B board used for ev-
ery H0–H4 replay (uncropped photograph).

Figure 3.6. Physical replay and exact-comparison setup. Each independently implemented H0–H4 bitstream was programmed on the same Nexys 3 board, replayed with the common request corpus, captured through UART, and compared field-by-field with the fixed-point reference; panel (b) shows the uncropped photograph of the board used. Only the accepted-request-to-done interval inside the solid FPGA kernel boundary contributes to reported kernel latency. Dashed host and transport functions, including programming, serialization, parsing, candidate generation, KMS/SDN access, and route installation, are excluded.

Pseudocode 3.3. Same-board replay and exact comparison

Input: independently built configuration h, common replay $\mathcal{D}$, expected records Y_h.
Output: raw capture, parsed table, mismatch report.

1. Program the bitstream for h and verify the expected configuration identity.
2. Present each replay request only when the shell is ready.
3. Retain the raw 44-byte response before parsing.
4. Parse all nine fields and reject missing or duplicate test identifiers.
5. Compare every integer field with Y_h; do not use numeric tolerances.
6. Retain cycle, status, profile, and selected-route summaries only after exactness passes.

Figure 3.1 and the transport specification define the implemented measurement boundary and retained physical interface.

3.19 Statistical Analysis

3.19.1 Training-level summaries

Controller evaluation uses four validation or test trace seeds as the paired unit. The registered training-level analysis uses 10,000 paired-seed percentile bootstrap replicates with RNG seed 32452843, exact paired permutation tests, and Holm adjustment for the planned comparisons. Final descriptive results are reported on the original reward scale. With four pairs, the smallest attainable two-sided exact permutation p-value is coarse; the reported raw value 0.125 and Holm-adjusted value 0.25 are therefore interpreted descriptively rather than as proof of superiority.

3.19.2 Physical route-quality summaries

The physical paired sampling unit is one jointly successful held-out replay row. For comparison $A - B$, difference $d_i = q_{i,A} - q_{i,B}$ is in UNORM16 code units, so positive means higher route quality in A. The paired standardized effect is

$$d_z = \frac{\bar{d}}{s_d}, \tag{3.22}$$

where s_d is the sample standard deviation of nonaggregated paired differences. The exact two-sided sign test discards zero differences and applies a binomial test to the remaining positive and negative signs. The two physical comparisons are treated as exploratory and their p-values are not multiplicity-adjusted.

The raw H2–H4 board rows make the interval calculation reproducible. For each comparison, the script filters held-out IDs 1000–1063, keeps the 58 jointly successful rows, and performs 10,000 paired-row bootstrap replicates with Python MT19937 seed 20260731. It reports percentile limits using R-7 linear interpolation at $p(B-1)$. The executable, source hashes, row differences, and summary preserve the complete calculation.

3.20 Supporting VLSI Methodology

The supplementary VLSI study compares isolated arithmetic and comparison kernels using Yosys generic synthesis and separate SKY130 OpenROAD flows through routing and report generation. Generic cells are not FPGA LUTs, and isolated kernel areas are not summed into a QFlow ASIC area.

FDPE-V3 is a supplementary LUT/interpolation experiment, not a fidelity-decay function in QFlow. The ngspice 2:1 multiplexer uses simplified Level-1 models at 1.8 V and 5–50 fF; its calculated $\frac{1}{2}CV^2$ applies only to that load and is not accelerator energy per decision.

3.21 Methodological Limitations

The study uses one synthetic Ring-6 environment, one FPGA family, and 84 records per configuration. Route quality is abstract, not a live measurement or stored-key state. No matched CPU benchmark, calibrated FPGA power, live QKD/KMS/SDN deployment, or route-installation timing was completed. Training uses α and ratio fields; the physical FPGA deploys a ratio-only projection over supplied cost. The isolated VLSI kernels do not constitute full-chip signoff.

3.22 Conclusion

The methodology specifies upstream eligibility and FPGA validity gating, H0–H4, the complete controller/reward/numeric contract, ratio-only physical deployment, cycle and protocol boundaries, and reproducible statistics. It keeps the bounded route-quality experiment distinct from classical key-resource semantics.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Introduction

This chapter reports software, RTL, post-route, and physical-board results in their respective evidence categories. The central result is a same-board comparison of five independent fixed-point candidate-evaluation configurations.

4.2 Experimental Overview

The evaluation followed four linked stages. Offline software training and held-out traces established controller behavior. Exhaustive and integrated RTL tests checked state encoding, policy lookup, dwell control, fixed-point arithmetic, and protocol timing. Independent Xilinx ISE builds provided post-route timing and resource data. Finally, the same Nexys 3 board replayed a common physical corpus for all five configurations.

The strongest comparison is internal Class A evidence: identical board, shell, clock, widths, record schema, and measurement boundary. Route-quality statistics are reported separately from exactness and implementation metrics.

4.3 Offline Controller Behavior

Each of the eight policy candidates completed 819,200 training transitions. Seven passed the validation criteria, and seed 229 was selected by the declared ranking. Table 4.1 summarizes the four-trace held-out software partition after the policy was fixed. All controllers completed 2,048 successful decisions with zero blocking in this controlled environment.

Table 4.1. Software-simulated held-out controller behavior over four 512-decision traces.

Controller	Mean evaluation reward	Block	Quality	Mean balance utility	Hops	Switch rate
Fixed-profile	4.6130	0	0.95815	0.16881	2.8750	0
Rule-adaptive	4.6292	0	0.95943	0.25513	2.9775	0.2813
Offline-learned adaptive	4.6771	0	0.96044	0.26499	2.9541	0.2329

All reward values in Table 4.1 are means of r_t^{eval}. The learned policy had the highest descriptive mean evaluation reward and the highest reported mean balance utility in this software

partition. Because $U_B = 1 - \mathrm{clip}_{[0,1]}(I_t^{\mathrm{post}})$, larger balance utility means lower post-decision imbalance under the defined utility. With only four paired traces, the exact two-sided permutation p-value was 0.125 and the Holm-adjusted value was 0.25 for both learned comparisons. The software result therefore describes behavior under the selected environment; it is not evidence of statistical superiority.

The learned mean of 4.677079 is an evaluation-scale mean of r_t^{eval}, not a training-target mean. It is interpreted using Equations (3.16)–(3.17). Zero blocking means each step received the $+4$ success term. The remaining approximately 0.677 mean contribution is the net of bounded route-quality and balance utilities, hop costs, and profile-switch penalties. It is not an end-to-end secret-key rate, monetary value, or physical energy measure. The route-quality column likewise reports the registered synthetic metric; it is not evidence of coherence in a stored classical key pool.

4.4 Controller and Policy Verification

Table 4.2. Controller and policy verification summary.

Verification item	Count/result	Outcome
Encoded state addresses	256/256	Exact policy/state mapping
Threshold below/equal/above cases	36	Equality entered the higher bin
Profile actions	4/4	All profiles covered
Cycle vectors	1,509	Zero RTL mismatch
Valid decisions	302	Zero unexplained mismatch
Python regression tests	74	Pass
Lint warnings	0	Pass
Controller-only synthesis estimate	103 LUTs, 79 registers	Synthesis pass

The controller-only synthesis reported an estimated maximum frequency of 247.812 MHz. This value is a post-synthesis estimate for the isolated controller and is not used as the system frequency. Final maximum-frequency results are taken from complete post-route implementations.

4.5 Integrated RTL Verification

Integrated cycle-accurate tracing added 2,125 cycle vectors and 530 valid decisions. The valid-request controller path had a deterministic three-cycle latency. Reset, busy, no-path, invalid-input, threshold equality, dwell hold, dwell switch, and pipeline-wait cases were exercised.

Candidate-evaluator regression covered 519 of 519 vectors, and the integrated policy shell matched 101 of 101 expected results. These are RTL-simulated results; physical exactness is established by the board replay below.

4.6 Independent FPGA Implementation

Configurations H0–H4 were built independently for the XC6SLX16-2-CSG324 device. Each build passed synthesis, mapping, placement, routing, and the 100 MHz timing constraint. Independent bitstreams prevent a parameter-only simulation from being reported as multiple physical implementations. Configuration H0 remained the feasible minimum-hop baseline, while configurations H1–H4 added the cost and profile-scoring functions defined in Table 3.4.

4.7 Same-Board Physical Replay and Exactness

Each configuration produced 84 physical records: 20 directed contract records and 64 held-out records. Nine output fields were checked per record. Table 4.3 reports the exact comparison.

Table 4.3. Physical replay exactness for the five independent configurations.

Configuration	Records	Fields/record	Exact fields	Mismatches
H0	84	9	756	0
H1	84	9	756	0
H2	84	9	756	0
H3	84	9	756	0
H4	84	9	756	0
Total	**420**	—	**3,780**	**0**

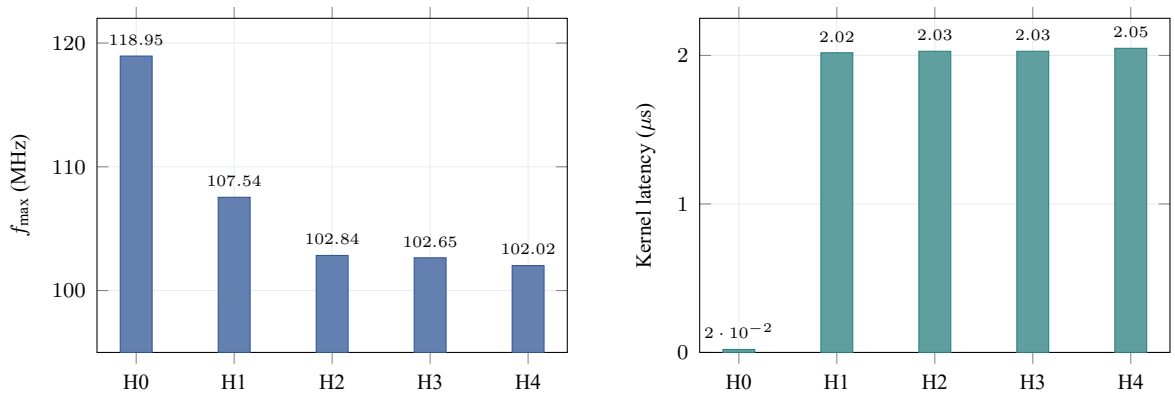
This result establishes that physical transport and parsing functioned, each independent implementation matched its fixed-point contract, and configuration H4 performed policy/profile inference consistently with the reference. Exactness does not by itself establish that one controller selects higher-quality routes.

4.8 Post-Route Timing and Kernel Latency

Table 4.4. Post-route maximum frequency and measured kernel latency under the common 100 MHz operating contract.

Configuration	Selection method	f_{max} (MHz)	Mean cycles	Latency (μs)
H0	Feasible minimum-hop baseline	118.948	2.000	0.020
H1	Fixed key-aware baseline	107.538	201.763	2.018
H2	Fixed-profile **QFlow**	102.838	202.763	2.028
H3	Threshold/rule adaptive	102.648	202.763	2.028
H4	Reinforcement-learned adaptive	102.020	204.763	2.048

All configurations exceeded 100 MHz. Configuration H4 achieved a post-route maximum frequency of 102.020 MHz and a mean normal-case latency of 2.047632 μs, rounded to 2.048 μs. This is the accepted-request-to-done latency of the tested FPGA kernel. UART serialization, host processing, KMS acquisition, candidate generation, security checks, and route installation are outside the measurement.



(a) Post-route maximum frequency.

(b) Mean normal-case latency.

Figure 4.1. Timing and latency across the H0–H4 comparison.

The reciprocal of the rounded configuration H4 kernel latency is $1/(2.048\ \mu\text{s}) \approx 488{,}281$ transactions/s. This is a theoretical sequential kernel-service rate under immediate back-to-back acceptance, not measured end-to-end throughput.

Configuration H0 is not feature-equivalent to the remaining configurations; its two-cycle result should not be used to calculate an RL overhead. The relevant adaptive comparison is configuration H4 versus configuration H3.

The timing separation follows directly from the implemented datapaths. H0 selects the valid candidate with the smallest hop count, using candidate slot as the deterministic tie-break, and registers completion without range normalization. H1–H4 use the shared iterative 49-by-33-bit

divider; objective ranges that are not equal require repeated division steps, so their median is near the 309–312-cycle maximum even though bypassed cases reduce the mean. H1 omits the profile-ROM read. H2 and H3 add that one setup cycle, and their common candidate evaluator therefore has the same measured mean. H4 adds two controller cycles for policy-ROM/dwell inference and retains the same scoring engine. The resulting H4–H3 difference is two cycles, or 20 ns at 100 MHz, while the full H0–H4 gap mainly measures the presence of the multi-objective evaluator rather than reinforcement learning.

4.9 Resource Utilization

Table 4.5. Post-route resource utilization of independent H0–H4 implementations.

Configuration	Slice registers	Slice LUTs	Occupied slices
H0	493	801	301
H1	1,000	1,484	589
H2	1,085	1,650	649
H3	1,097	1,836	645
H4	1,156	1,706	677

Configuration H4 used 1,706 LUTs, 1,156 registers, and 677 occupied slices. Relative to configuration H3, it mapped to 7.081% fewer LUTs, 5.378% more registers, and 4.961% more occupied slices. LUT count and occupied slices can move in opposite directions because slice packing, register placement, and routing affect physical occupancy. The LUT reduction is an implementation-specific synthesis result, not a general property of reinforcement learning. Figure 4.2 plots the three resource categories on separate zero-based axes, making the H3–H4 mapping trade-off visible without combining unlike resources on one scale.

Table 4.6. Detailed H4 utilization from the Xilinx map report for XC6SLX16-2-CSG324. Percentages are the report's integer values.

Resource	Used	Available	Utilization
Slice registers	1,156	18,224	6%
Slice LUTs	1,706	9,112	18%
Occupied slices	677	2,278	29%
Bonded I/O blocks	17	232	7%
RAMB16 block RAMs	0	32	0%
RAMB8 block RAMs	0	64	0%
BUFG global clock buffers	1	16	6%
DSP48A1 slices	0	32	0%

The zero BRAM and DSP counts in Table 4.6 are explicit tool-report entries, not assumptions from missing categories. Policy and profile storage mapped into distributed logic for this small design. Occupied-slice capacity also explains why 677 slices correspond to the report's 29% even though individual LUT and register percentages are lower.

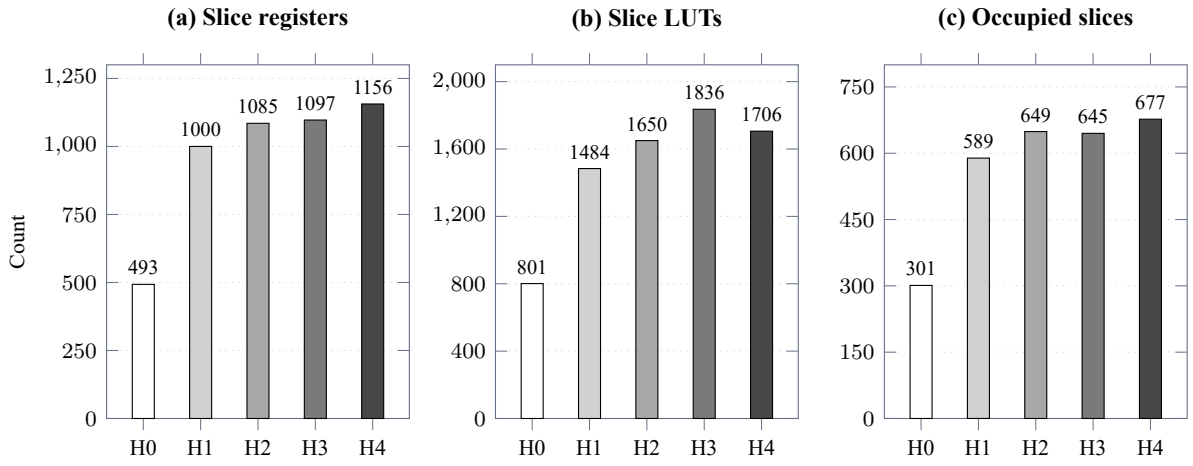
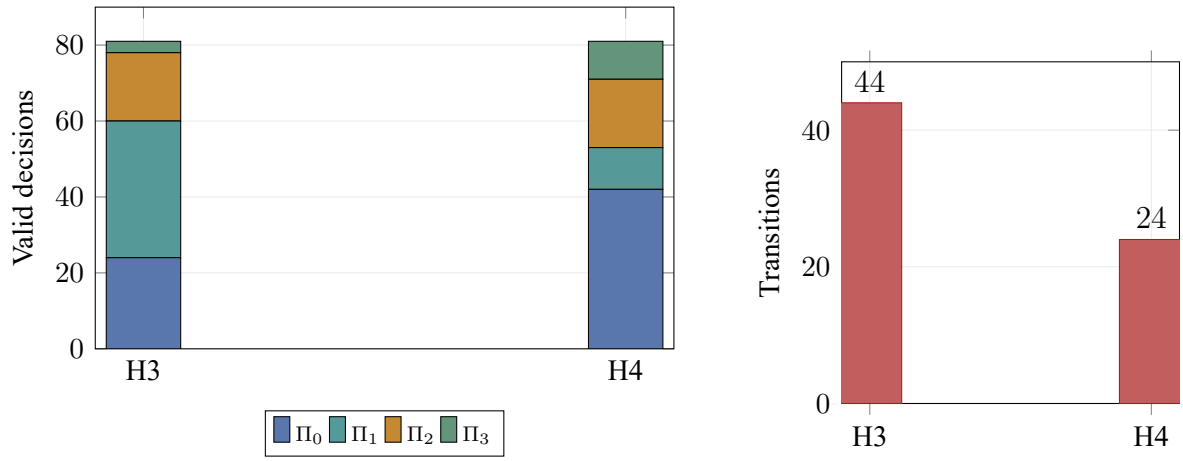


Figure 4.2. Post-route FPGA resource utilization for the five independent configurations. The aligned, zero-based panels report exact slice-register, slice-LUT, and occupied-slice counts from Table 4.5; grayscale shade identifies H0 through H4 consistently in each panel. H4 mapped to fewer LUTs than H3 but used more registers and occupied slices, so individual resource categories and physical slice packing need not move in the same direction.

4.10 Adaptive Profile Behavior

Table 4.7. Profile use and switching over 81 valid adaptive decisions.

	Profile count				Transitions	Runs	Min dwell	Max dwell
Configuration	Π_0	Π_1	Π_2	Π_3				
H3	24	36	18	3	44	45	1	7
H4	42	11	18	10	24	25	1	7



(a) Profile distribution.

(b) Profile transitions.

Figure 4.3. Trace-specific adaptive behavior of configurations H3 and H4.

The complete configuration H4 controller, including the offline-learned policy ROM and minimum-dwell mechanism, reduced observed profile transitions from 44 to 24. This is a 45.455% reduction on the evaluated replay, while mean dwell increased from 1.80 to 3.24 valid decisions. Isolating the learned policy contribution would require a policy-by-dwell ablation.

The denominator 81 is the number of protocol-valid adaptive decisions in the full 84-row corpus. Records with test IDs 4, 1015, and 1047 were deliberately malformed and returned `INVALID_REQUEST`; they were excluded because no valid controller decision occurred. The five `NO_PATH` records (IDs 3, 1007, 1023, 1039, and 1055) remained included: they carried valid state inputs, advanced the profile/dwell controller, and failed only because no candidate was feasible. Thus the profile counts sum to 81 for both H3 and H4.

4.11 Held-Out Route-Quality Analysis

All configurations had 58 successful, four no-path, and two invalid rows in the 64-row held-out physical partition. Upstream candidate validity and the common request fields determine

those outcomes before profile-conditioned soft preference; the FPGA gates supplied validity rather than recomputing link eligibility. Consequently, a mode could not convert a deliberately invalid row into a valid request or select a route when both candidates were infeasible. Differences appeared only in which feasible candidate was preferred. Table 4.8 therefore compares configuration H4 with H2 and H3 only on the 58 jointly successful rows.

Table 4.8. Paired held-out physical route-quality comparisons.

Comparison	Pairs	Mean diff.	95% CI	p	d_z	Path changes
H4–H2	58	86.47	$[-25.12, 226.62]$	0.2188	0.176	6
H4–H3	58	-78.05	$[-242.28, 77.53]$	0.4531	-0.127	7

The route-quality field is unsigned UNORM16: valid codes are 0–65,535, larger is better, and one unit is $1/65535 \approx 0.000015259$ of full scale. The exact mean H4–H2 difference of $+86.465517$ units equals $+0.00131938$ normalized, or about $+0.132$ percentage points of full scale. The H4–H3 difference of -78.051724 units equals -0.00119099, or about -0.119 percentage points. These effects are small relative to the complete scale. In H4–H2, 5 pairs were higher, 1 lower, and 52 tied; in H4–H3, 2 were higher, 5 lower, and 51 tied.

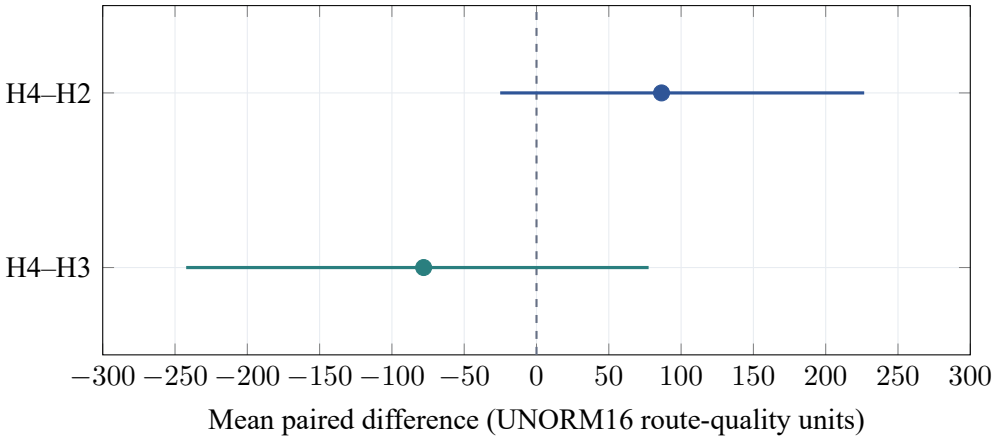


Figure 4.4. Paired mean route-quality differences with 10,000-replicate paired-row percentile 95% confidence intervals (seed 20260731). Both intervals cross zero; the line at zero denotes no mean difference.

Both confidence intervals cross zero. The p-values are 0.2188 for configuration H4 versus configuration H2 and 0.4531 for configuration H4 versus configuration H3. The limited number of rows and possible dependence among replay rows reduce statistical power. Configuration H4 demonstrated physical deployability and distinct adaptive behavior, but the available held-out physical comparisons do not establish statistically significant route-quality superiority over configurations H2 or H3.

4.12 H4 Versus H3 Engineering Trade-Off

Table 4.9. Configuration H4 relative to rule-adaptive configuration H3.

Metric	Change	Interpretation
Mean normal-case latency	+0.986%	Small additional policy/control latency
Post-route $f_{\max}$	−0.612%	Both configurations remain above 100 MHz
Slice LUTs	−7.081%	Implementation-specific mapping result
Slice registers	+5.378%	Additional registered control/state
Occupied slices	+4.961%	Increased physical footprint after packing/routing
Profile transitions	−45.455%	Complete policy-and-dwell controller switched less
Mean route-quality code	−78.05 units	Small, non-significant paired difference

The engineering contribution of configuration H4 is a bit-exact ratio-only deployed control law with microsecond-scale latency and explicit physical costs. Because physical path cost was supplied and common across profiles, this exactness does not establish equivalence to the richer training action in which profile-dependent α coefficients also construct path cost. Its value does not depend on winning every route-quality comparison.

4.13 Comparison with Related Work

Table 4.10 compares evidence dimensions without constructing speedup ratios across different functions.

Table 4.10. Function- and measurement-aware comparison with representative verified work. "Direct" requires the same evaluated function, input contract, and timing boundary; only the internal H0–H4 rows satisfy that requirement.

Work/category	Evaluated function	Platform	Evidence	Evaluation scope	HW	Board	Bit exact	Direct
Bi et al. [5]	Dynamic key-resource routing	Software	Network model	Simulated QKD topology	No	No	No	No
Bhatia et al. [22]	Security-aware route/wavelength/time-slot allocation	Software	Network simulation	NSFNET and UBN24 experiments	No	No	No	No
Hao et al. [8]	Resource-aware tabular Q-learning routing	Software	Network simulation	Trusted-relay model	No	No	No	No
Johann et al. [9]	DRL routing across flexible mesh sizes	Software	Network simulation	Multiple meshed topologies	No	No	No	No
Li et al. [25]	Progressive QKD-network recovery	Software DRL	Failure-recovery simulation	Staged repair decisions	No	No	No	No
Popa and Popescu [24]	KMS-level key forwarding	Linear program	Network optimization	All-to-all/one-to-all/one-to-one cases	No	No	No	No
MadQCI [3]	SDN-QKD integration	Distributed system	Operational deployment	Multi-site production facilities	No	No	No	No
Venkatachalam et al. [11]	QKD post-processing	FPGA	Component implementation	Post-processing workload	Yes	Yes	Different	No
Branchini et al. [33]	Quantum-error-correction decoder requirements	FPGA survey	KPI synthesis	Quantum-adjacent decoder scope	Yes	Various	Different	No
QFlow H0–H4	Fixed-point route-candidate evaluation	Spartan-6 FPGA	Post-route and physical replay	420 records across five independent builds	Yes	Nexys 3	Yes	Internal only

QFlow is stronger in physically validated fixed-point candidate-evaluation evidence, while several software studies provide broader topology, network-level, or generalization evidence. Direct numerical comparison is limited to the internal same-board configurations.

4.14 Practical Deployment Scenario

A realistic deployment would partition responsibilities as follows:

1. The KMS/SDN controller receives key-pool occupancy, effective key-generation rate, link activity, QBER-derived or other externally measured health indicators, request load, and utilization.
2. The controller generates route candidates, computes hard eligibility from the current topology, resources, health, and policy, and assigns each validity bit.
3. Candidate descriptors, including those validity bits and precomputed path costs, plus state metadata are sent to **QFlow**.
4. **QFlow** encodes the state, selects profile Π_k, evaluates the candidates, and returns a candidate identifier, profile, score, and status.
5. The controller checks administrative and security policy, installs the selected route, and handles failure or fallback.

The 2.048 μs value applies only to step 4 within the tested FPGA boundary. It is not the time to collect KMS data or install a network route. The practical value of the accelerator lies in deterministic recurring computation, fixed-point reproducibility, predictable kernel latency, and reconfigurable profile deployment.

4.15 Platform Selection and Cost Implications

4.15.1 CPU role

CPU/software remains preferable for topology management, KMS and SDN orchestration, candidate generation, policy management, offline Q-learning, logging, security administration, and irregular or infrequently executed tasks. No same-computation, same-input CPU timing experiment was completed. Accordingly, this thesis reports no CPU-to-FPGA speedup and no CPU/FPGA energy comparison.

4.15.2 FPGA role

The measured configuration H4 evidence comprises 102.020 MHz post-route maximum frequency, 2.048 μs mean kernel latency, 1,706 LUTs, 1,156 registers, 677 occupied slices, and

exact hardware/reference agreement. FPGA implementation was appropriate for the present research stage because the architecture and profiles remain evolvable, bitstreams can be changed after deployment, configurations H0–H4 can be implemented on the same device, deterministic pipelines are available, and physical implementation evidence can be collected before committing to silicon.

4.15.3 ASIC role

A mature ASIC could potentially provide lower energy per transaction, higher density, greater performance, and lower high-volume unit cost. It would require substantial non-recurring engineering, complete signoff, fabrication, packaging, testing, longer development time, stronger verification, redesign cost when the model changes, and loss of post-fabrication reconfigurability.

For the present low-volume, evolving research prototype, FPGA is the more appropriate engineering platform. For a stable, high-volume commercial product, ASIC implementation may eventually be more economical and energy-efficient.

4.15.4 Conceptual total-cost model

Let production volume be N. A simple platform cost model is

$$C_{\text{FPGA}}(N) = C_{\text{dev,FPGA}} + N c_{\text{FPGA}}, \tag{4.1}$$

$$C_{\text{ASIC}}(N) = C_{\text{NRE,ASIC}} + N c_{\text{ASIC}}, \tag{4.2}$$

where $C_{\text{dev,FPGA}}$ is FPGA development and integration cost, $C_{\text{NRE,ASIC}}$ includes ASIC design, verification, masks, fabrication, packaging, and test, and c_{FPGA} and c_{ASIC} are per-unit costs. When $c_{\text{FPGA}} > c_{\text{ASIC}}$, the conceptual break-even volume is

$$N^{\star} = \frac{C_{\text{NRE,ASIC}} - C_{\text{dev,FPGA}}}{c_{\text{FPGA}} - c_{\text{ASIC}}}. \tag{4.3}$$

Below $N^{\star}$, FPGA can have lower total cost; above it, a stable ASIC may become economical. The thesis does not estimate $N^{\star}$ because verified commercial cost inputs are unavailable.

Table 4.11. Qualitative platform comparison for the **QFlow** development stages.

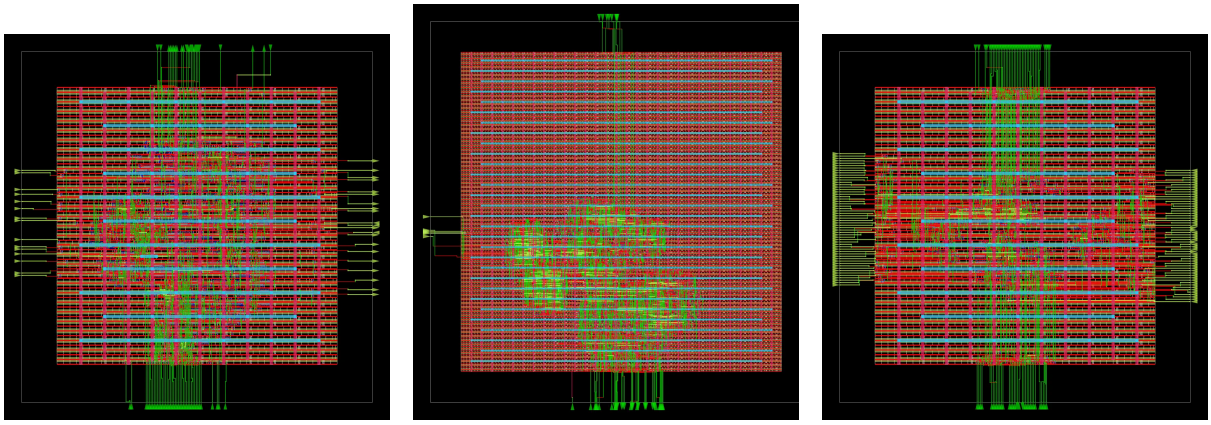
Criterion	CPU	FPGA	ASIC
Development effort	Low to moderate	Moderate	High
Reconfigurability	Software update	Bitstream update	None after fabrication
Deterministic execution	Requires careful system design	Natural for fixed pipeline	Natural for fixed design
Low-volume total cost	Usually favourable	Favourable for hardware prototype	Usually unfavourable
High-volume unit cost	Processor/platform dependent	Often higher	Potentially lowest
Energy potential	General-purpose overhead	Specialized datapath potential	Highest specialization potential
Time to deployment	Short	Moderate	Long
Current **QFlow** suitability	Control plane and training	Candidate-evaluation prototype	Premature
Future mature **QFlow**	Orchestration and fallback	Reconfigurable product option	Potential high-volume option

4.16 Kernel-Level Physical-Design Exploration

The supporting VLSI study examined selected computation kernels rather than a complete accelerator. Yosys generic synthesis reduced the SKAG scoring kernel from 5,343 cells with runtime products to 503 cells with fixed-coefficient shift-add logic, a 90.59% reduction within that flow. This result illustrates the structural benefit of coefficient specialization; generic cells are not FPGA LUTs.

Table 4.12. OpenROAD physical-design results for isolated kernels.

Kernel	Evaluated role	Area (μm^2)	Utilization
SKAG-W1	Fixed-coefficient shift-add scoring	6,451	26%
LUT/interpolation arithmetic kernel (repository identifier FDPE-V3)	Supplementary fixed-point interpolation study	22,186	13%
Pareto-C0	Dominance and selection comparator	3,368	13%



(a) SKAG-W1. **(b)** LUT/interpolation kernel (FDPE-V3). **(c)** Pareto-C0.

Figure 4.5. Routed OpenROAD layouts for three isolated kernels. These views do not represent an integrated **QFlow** ASIC.

The flows generated routed reports and final artifacts for the separate kernels, showing that selected functions can pass through an open-source physical-design workflow. The areas must not be summed as a **QFlow** chip area. The study does not provide full-chip power, commercial DRC/LVS signoff, tapeout, or fabricated-silicon performance.

4.17 Threats to Validity

- The physical experiment used one FPGA board and device family.
- The physical corpus contained 84 rows per configuration and a bounded candidate set.
- The offline-learned policy was evaluated in a controlled research environment rather than across many independent unseen topologies.
- Replay rows may share structure, reducing the effective independence and power of route-quality tests.
- No matched CPU benchmark, activity-calibrated power measurement, or live KMS/SDN deployment was available.
- Configuration H4 performed inference only; online learning was not tested.
- Training used profile-dependent α coefficients and objective ratios, whereas the physical replay deployed only the ratio projection over supplied profile-independent path costs.
- The OpenROAD results concern isolated kernels and do not establish a complete ASIC implementation.

4.18 Answers to the Research Questions

Table 4.13. Answers to the research questions.

RQ	Answer	Evidence
RQ1	All five independent configurations exceeded the 100 MHz post-route target.	Table 4.4
RQ2	All 420 records and 3,780 checked fields matched exactly, with zero mismatch.	Table 4.3
RQ3	Configurations H1–H4 required more logic and cycles than minimal configuration H0; configuration H4 added small timing changes relative to H3.	Tables 4.5 and 4.9
RQ4	Configuration H4 used all four profiles and the complete controller produced 45.455% fewer transitions than configuration H3 on the replay.	Table 4.7
RQ5	The held-out comparisons did not establish statistically significant route-quality improvement.	Table 4.8

4.19 Research-Hypothesis Outcomes

Table 4.14. Mapping from research hypotheses to evidence and outcomes.

Hypothesis	Evidence	Outcome	Interpretation
RH1	Five post-route maximum-frequency reports	Supported	Every configuration exceeded 100 MHz
RH2	3,780 field comparisons	Supported	Zero physical/reference mismatch
RH3	2.048 μs H4 latency and implementation metrics	Supported	Microsecond-scale inference with measured resource/timing trade-offs
RH4	44 versus 24 transitions	Supported	Complete H4 policy-and-dwell controller switched less on the common replay
RH5	Paired CIs and p-values 0.2188 and 0.4531	Not supported	Route-quality superiority was not established

Chapter conclusion. RH1–RH4 are supported; RH5 is not. The FPGA evidence establishes exact, microsecond-scale adaptive behavior, but neither route-quality superiority nor an integrated ASIC.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Thesis Summary

This thesis developed and physically evaluated **QFlow**, a fixed-point FPGA accelerator for adaptive key-resource-aware route-candidate evaluation in trusted-relay QKD networks. The system accepts externally generated candidate descriptors, gates each candidate using its upstream-computed validity bit, selects a fixed or adaptive scoring profile, evaluates candidates with deterministic arithmetic, and returns a selected identifier and status. Candidate generation, hard-eligibility computation, topology management, KMS operation, SDN orchestration, security validation, and route installation remain control-plane functions.

The adaptive controller quantizes four features into a 256-state address and selects one of four scoring profiles. Tabular Q-learning is performed offline. The deployed reinforcement-learned configuration uses a 256-entry two-bit policy ROM and minimum-dwell control, and performs deterministic inference without runtime Q-value updates.

The software-training action used the complete registered profile: four α coefficients constructed profile-dependent path costs and three Tchebycheff ratios weighted the objectives. In the physical experiment, path costs were supplied precomputed and common across profiles, so only the ratios affected selection. The physical exactness result therefore validates this ratio-only deployment projection, not a complete on-chip realization of the training cost function.

The executed features are minimum key occupancy, bounded bottleneck route quality, offered request load, and utilization imbalance. The second feature is limited to its role as an abstract experimental input and is not a stored-classical-key coherence quantity. This experimental contract is kept separate from the final network model based on occupancy, generation, consumption, status, utilization, freshness policy, and external health indicators.

5.2 Main Findings

Five independent hardware evaluation configurations were implemented on a Digilent Nexys 3 Spartan-6 FPGA under a common interface, clock, replay, and measurement contract. All configurations exceeded the 100 MHz post-route target. The physical campaign produced 420 records and 3,780 checked fields, all of which matched the fixed-point reference exactly.

Configuration H4 achieved a post-route maximum frequency of 102.020 MHz and a mean normal-case accepted-request-to-done kernel latency of 2.048 μs. Relative to configuration H3, configuration H4 changed mean latency by +0.986% and maximum frequency by -0.612%, mapped to 7.081% fewer LUTs, 5.378% more registers, and 4.961% more occupied slices. These values describe one implemented mapping and do not imply a universal resource property

of reinforcement learning.

The complete configuration H4 controller, including the learned policy ROM and minimum-dwell mechanism, produced 24 profile transitions compared with 44 for configuration H3, a 45.455% reduction on the common replay. The corresponding route-quality comparisons were not statistically significant: the configuration H4 versus H2 and H4 versus H3 sign-test p-values were 0.2188 and 0.4531, and both confidence intervals crossed zero. The evidence therefore supports physical deployability, exactness, and distinct adaptive behavior, but does not establish route-quality superiority.

The supporting VLSI study showed that selected kernels can pass through generic synthesis and separate OpenROAD flows. Fixed-coefficient specialization reduced one scoring kernel from 5,343 to 503 generic cells, while isolated routed areas were 6,451 μm^2 for SKAG-W1, 22,186 μm^2 for the LUT/interpolation arithmetic kernel (repository identifier FDPE-V3), and 3,368 μm^2 for Pareto-C0. These results indicate a possible future ASIC path; they are not an integrated ASIC implementation.

5.3 Contributions

The work contributes:

1. a classical key-resource model for candidate evaluation that represents pool occupancy, generation, consumption, status, utilization, health indicators, and policy-controlled freshness;
2. an upstream hard-eligibility contract and bounded fixed-point FPGA evaluator that gates supplied validity before soft multi-objective preference;
3. fixed, rule-adaptive, and offline-learned profile-selection mechanisms;
4. a deterministic policy-ROM deployment architecture with minimum-dwell control;
5. five independent FPGA configurations and a common physical measurement contract; and
6. reproducible exactness, timing, latency, resource, adaptive-behavior, and neutral route-quality evidence.

The principal contribution is thus a reproducible hardware-systems result rather than a claim of universal routing superiority.

5.4 Limitations

The candidate evaluator was tested on a bounded record format, one FPGA board and device family, and 84 records per configuration. The replay establishes exact implementation behavior for those inputs but does not establish network-level superiority on unseen topologies or live

QKD deployments. The policy was trained in a controlled research environment, and replay rows may contain dependence that reduces the effective statistical sample size.

The richer software-training action and ratio-only physical deployment are not functionally identical: the physical shell did not recompute profile-conditioned path cost. A future implementation should either move that cost construction into the hardware boundary or train and evaluate explicitly against the same profile-independent supplied costs used for deployment.

No exactly matched CPU/FPGA timing benchmark, activity-calibrated FPGA power measurement, or CPU/FPGA energy comparison was completed. No live KMS, SDN controller, or QKD device was integrated, and the host/FPGA link was an experimental transport rather than a production authenticated interface. The VLSI results concern separate kernels and do not provide integrated area, full-chip power, foundry signoff, tapeout, or fabricated performance.

5.5 Future Work

The highest-priority next step is an exactly matched CPU/FPGA benchmark using the same candidate descriptors, fixed-point contract, request protocol, and timing boundary. This would support a defensible kernel speed comparison and expose the effect of software scheduling and data transfer.

Second, FPGA power and energy should be measured with activity-calibrated switching data and a defined operating point for every configuration. Energy per accepted candidate-evaluation transaction should be separated from board idle power and host transport.

Third, the policy should be tested on larger independent trace sets and unseen topologies. The evaluation should include stronger RL and non-RL baselines, bursts, key-pool depletion and recovery, failures, changing key-generation rates, and statistically powered route-quality analysis.

Fourth, a policy-by-dwell ablation should independently vary the policy and minimum-dwell mechanism. This would determine how much of the observed switching reduction arises from the learned mapping, the dwell rule, and their interaction.

Fifth, **QFlow** should be integrated with a live KMS/SDN testbed. The integration must measure metadata acquisition, candidate generation, host/FPGA communication, security-policy checks, fallback, and route installation separately from kernel latency.

Sixth, the host/FPGA interface should be authenticated and hardened. Input validation, replay protection, policy identity, signed bitstreams where supported, fault containment, audit logging, and software fallback are important for a security-sensitive controller.

Finally, an integrated full-core ASIC study can be considered after the mathematical and interface contracts stabilize. Such work would require complete logic and physical verification, clock and power design, timing closure, DRC/LVS, packaging, test planning, and a measured comparison with the FPGA implementation.

5.6 Closing Statement

The experiments show that the ratio-only projection of an offline-learned key-resource-aware controller can be deployed bit-exactly on a resource-constrained FPGA with deterministic microsecond-scale candidate-evaluation latency and controlled implementation overhead. Equally important, the neutral route-quality result defines where the evidence stops. This combination of physical exactness, transparent trade-offs, and bounded interpretation provides a sound basis for larger network experiments and future deployment studies.

CHAPTER A
REPRODUCIBILITY AND ARTIFACT REGISTER

A.1 Repository snapshots

Repository:

https://github.com/rizvi19/thesis-qcnfpga

Primary FPGA and controller evidence:

branch `rl-nexys3-adaptive`,

commit `290b4ad75eef2af2b9da2f1f281a2b89416cfd24`.

Supporting VLSI evidence:

branch `vlsi-kernel-study`,

commit `0093ba288f26a1520e4318f207e13f52d38db5a8`.

Submission source:

branch `thesis-final-submission-corrections`,

commit `f5d665d083fc3ed7f3119a676acf5d236a3d5475`.

The submission-source identifier records the source commit created before the release PDF and packaging artifacts. The later packaging commit does not redefine the source snapshot.

A.2 Primary artifact register

Table A.1. Primary reproducibility artifacts.

Artifact	Repository location
RL model description	`docs/rl/rl_model_card.md`
Policy selection	`results/rl/p3_revision/selection_record.json`
Policy freeze	`results/rl/p3_policy_freeze/freeze_record.json`
Held-out controller results	`results/rl/p3_heldout/`
Controller verification	`results/rl/p4_exhaustive/` and `results/rl/p4_integrated/`
Integration checkpoint	`results/nexys3/p5_step9_final_checkpoint/`
Independent implementation reports	`results/nexys3/p6_step3_h0_ise/` through `results/nexys3/p6_step8_h4_ise/`
Physical records and comparisons	`results/nexys3/p6_step9_cross_mode/`

Artifact	Repository location
Submission-ready physical summaries	`results/nexys3/p6_step10_closure/`
Recomputed physical bootstrap	`reproduce_physical_bootstrap.py`, `data/physical_bootstrap_recomputed.csv`, and `data/physical_paired_differences.csv`
VLSI kernel results	`results/partC_vlsi/`
CMOS primitive results	`results/partD_cmos/`
Literature verification	`literature/p1/audit/` and `literature/p1/corrected/`

A.3 Headline numerical sources

Table A.2. Authoritative sources for headline numerical results.

Reported quantity	Source artifact
Physical records, exact fields, configuration H4 summary, and H4/H3 percentages	`results/nexys3/p6_step10_closure/final_summary.json`
H0–H4 post-route timing, cycles, resources, and exactness	`results/nexys3/p6_step10_closure/thesis_ready_same_board_table.csv`
Profile counts and transitions	`results/nexys3/p6_step10_closure/thesis_ready_adaptation_table.csv`
Paired route-quality means, signs, and effect sizes	raw mode rows under `results/nexys3/p6_step6_h2_physical/`, `results/nexys3/p6_step7_h3_physical/`, and `results/nexys3/p6_step8_h4_physical/` at the primary evidence commit
Paired percentile confidence intervals	`reproduce_physical_bootstrap.py` and `data/physical_bootstrap_recomputed.csv` (10,000 replicates; seed 20260731)
Integration timing inventory	`results/nexys3/p5_step9_final_checkpoint/p5_timing_inventory.csv`
SKAG generic synthesis	`asic/skag_weight_kernel/results/skag_yosys_comparison.csv`
Other kernel generic synthesis	`asic/fdpe_kernel/results/fdpe_yosys_comparison.csv` and `asic/pareto_cmp_kernel/results/pareto_yosys_comparison.csv`
OpenROAD area and utilization	the three `logs/6_report.log` files below `results/partC_vlsi/openroad_<kernel>/`
Multiplexer delay and load calculation	`results/partD_cmos/pareto_cmp_primitive/load_sweep/tg_mux_load_sweep.csv`

Reported quantity	Source artifact
Literature-search date, queries, and dispositions	`literature/p1/audit/search_log.csv`, `paper_audit.csv`, and `literature/p1/corrected/PROVENANCE.md`

A.4 Evidence classification

Table A.3. Evidence types used in the thesis.

Evidence type	Interpretation
Physical replay	Output captured from a programmed Nexys 3 FPGA
Post-route implementation	Timing and resources after placement and routing
RTL simulation	Cycle-accurate behavior of the register-transfer-level design
Software simulation	Reference-model, training, and network/controller output
Analytical result	Quantity calculated from a stated equation and inputs
Supporting physical design	OpenROAD result for an isolated kernel

A.5 Tool versions

The FPGA builds used Xilinx ISE 14.7. The final thesis was built with XeTeX 3.141592653-2.6-0.999995 and BibTeX 0.99d from MiKTeX 24.1. The retained OpenROAD reports identify OpenROAD v2.0-19993-ga613c7e53. Remaining Python, Icarus Verilog, Yosys, ngspice, and environment details are recorded by the version-controlled build scripts and logs associated with each artifact family.

A.6 Build and replay sequence

1. check out the branch and commit associated with the selected evidence set;
2. verify source, ROM, and replay hashes;
3. run the software reference and controller regression tests;
4. compile and run unit and integrated RTL simulations;
5. build one selected hardware configuration with Xilinx ISE 14.7;
6. inspect synthesis, map, placement-and-route, and timing reports;
7. program the volatile FPGA bitstream;

8. run the common UART replay and retain the raw response;
9. parse the response and compare all expected fields; and
10. generate cross-configuration tables only after every configuration passes exactness.

CHAPTER B
FIXED-POINT, STATE, AND PROTOCOL CONTRACT

This appendix collects the bit-level constants required to reproduce the executed controller without inferring them from prose.

B.1 State and Policy Storage

Table B.1. Adaptive-controller storage layout.

Storage	Depth	Useful width	Ordering/use
Policy ROM	256	2 bits	State IDs 0–255; deployed H4 actions
Profile ROM	4	18 bits	Profiles Π_0–Π_3; deployed payloads
Training Q-table	1,024	16 bits	State-major/action-minor signed Q8.7; not deployed

The radix-4 address is

$$z = (((b_K \times 4) + b_Q) \times 4 + b_L) \times 4 + b_I. \tag{B.1}$$

The two-bit fields occupy [7:6], [5:4], [3:2], and [1:0] in that order. The deployed policy contains action counts $(145, 43, 44, 24)$ for $(\Pi_0, \Pi_1, \Pi_2, \Pi_3)$.

B.2 Threshold Edge Contract

Table B.2. Real and encoded threshold constants.

Feature	Threshold 1	Threshold 2	Threshold 3	UNORM16 codes
Minimum key occupancy	0.25	0.50	0.75	16,384; 32,768; 49,151
Bottleneck route quality	0.90	0.93	0.96	58,982; 60,948; 62,914
Offered request load	0.25	0.50	0.75	16,384; 32,768; 49,151
Utilization imbalance	0.125	0.25	0.50	8,192; 16,384; 32,768

The four-bin function is left-closed at every threshold: equality enters the higher bin. Testing below, at, and above every one of the 12 thresholds gives 36 directed threshold-edge cases.

B.3 Numeric Formats and Special Codes

Table B.3. Fixed-point formats used by the physical candidate shell.

Quantity	Format	Scale/range	Rule
State and route quality	16-bit UNORM16	0–$65535 \leftrightarrow 0$–1	Round half up and clip
Path cost	32-bit UQ16.16	0–$2^{16} - 2^{-15}$ finite	Maximum finite `0xFFFFFFFE`; `0xFFFFFFFF` is reserved infinity; finite overflow saturates to the maximum finite code
Candidate utilization	32-bit UQ16.16	0–$2^{16} - 2^{-16}$	Full unsigned range through `0xFFFFFFFF`; upstream saturation; no reserved infinity code
Hop count	3-bit unsigned	0–7	Integer edge count
Candidate slot	2-bit unsigned	0–3	Stable final tie-break
Tchebycheff ratio	2-bit unsigned	0–3	Common scaling omitted
Weighted score	18-bit unsigned	0–262143	Maximum of three products
Key count	16-bit unsigned integer	Trace capacity 16	Synthetic environment
Synthetic key rate	16-bit UQ8.8	0–$2^{8} - 2^{-8}$	Zero makes a software path infeasible; positive values follow the software/export contract
α coefficient	3-bit U2.1	Half-unit increments	Stored in profile word

For nonconstant range $m < M$, normalization is

$$N(x) = \text{round}_{\text{half up}} \left(\frac{(x - m)65535}{(M - m) + 1} \right). \tag{B.2}$$

Equal ranges map to zero. The comparator ranking is $(\text{score}, \text{path cost}, \text{hops}, \text{slot})$, all ascending.

B.4 Profile Payloads

The 18-bit packing order is

$$\alpha_1[2{:}0]\ \alpha_2[2{:}0]\ \alpha_3[2{:}0]\ \alpha_4[2{:}0]\ \rho_c[1{:}0], \rho_q[1{:}0], \rho_u[1{:}0].$$

The payloads are `0x13329`, `0x1B516`, `0x14399`, and `0x0A2A5`. During software training, bits $[17{:}6]$ provide the four α coefficients used to construct profile-dependent path cost and bits $[5{:}0]$ provide the three objective ratios. In the physical candidate evaluator, path cost is supplied precomputed and common across profiles, so only bits $[5{:}0]$ affect selection. Thus the physical contract deploys the ratio projection of the full training profile; it does not evaluate the α-dependent cost function.

B.5 H3 Priority Function

For completeness, the exact first-match function is

$$a_{H3}(z) = \begin{cases} 1, & b_K = 0, \\ 2, & b_K \neq 0 \wedge b_Q \leq 1, \\ 1, & b_K \neq 0 \wedge b_Q > 1 \wedge b_I \geq 2, \\ 3, & b_L = 3 \wedge b_K \geq 2 \wedge b_Q \geq 2 \wedge b_I \leq 1, \\ 0, & \text{otherwise.} \end{cases} \tag{B.3}$$

The order is significant; the mathematical guards make the priority explicit.

B.6 Synchronous and UART Protocol

A request is accepted only on `start && ready`. Start while busy is ignored and does not alter the active request. Reset clears busy/done, cycle count, winner state, profile history, and dwell state. Done is a one-cycle pulse with all result fields stable.

Table B.4. Status and failure-result contract.

Code	Name	Returned fields
0	`OK`	Selected slot 0–3, computed score and route-quality code
1	`NO_PATH`	Selected path 255; score and route quality zero; valid controller state/profile retained
2	`INVALID_REQUEST`	Selected path 255; score and route quality zero; state/profile zero

The 44-byte record is

`QF6R,1,TTTT,P,SSS,R,PPP,CCCCCC,FFFFF,LLL,E\r\n`.

The nine numeric fields are test ID, policy ID, state ID, profile ID, selected path, score, route-quality code (registered field name bottleneck fidelity), cycles, and status. Production UART is 100 MHz/115,200 baud, 8N1, idle high, no flow control. Bytes are emitted left to right; each byte sends least-significant data bit first.

B.7 Latency Boundary and Observed Ranges

Cycle count is $N_{\text{done}} - N_{\text{accept}}$. H0 normal results are exactly two cycles. H1 normal results span 7–309 cycles, H2/H3 span 8–310, and H4 spans 10–312. Mean values are 2.000, 201.763, 202.763, 202.763, and 204.763 cycles. Host parsing, UART serialization, operating-system scheduling, JTAG, programming, KMS access, candidate generation, and route installation are excluded.

CHAPTER C
COMPLETE ADDITIONAL RESULTS

C.1 Additional Same-Board Route-Quality Results

Table C.1. Held-out status and route-quality summaries.

Configuration	Selection method	Rows	Success	No path	Invalid	Mean quality	Median quality
H0	feasible minimum-hop baseline	64	58	4	2	61707.19	62087.50
H1	fixed key-aware	64	58	4	2	61640.03	61673.50
H2	fixed-profile **QFlow**	64	58	4	2	61640.03	61673.50
H3	threshold/rule adaptive	64	58	4	2	61804.55	61673.50
H4	reinforcement-learned adaptive	64	58	4	2	61726.50	61837.00

C.2 Adaptive Profile Percentages

Table C.2. Adaptive profile use over 81 valid decisions.

Configuration	Π_0	Π_1	Π_2	Π_3
H3	29.630%	44.444%	22.222%	3.704%
H4	51.852%	13.580%	22.222%	12.346%

C.3 Controller Overhead Relative to Configuration H0

Table C.3. Absolute resource and cycle differences relative to minimal configuration H0.

Configuration	Δ registers	Δ LUTs	Δ slices	Δ normal cycles
H1	507	683	288	199.763
H2	592	849	348	200.763
H3	604	1,035	344	200.763
H4	663	905	376	202.763

These differences quantify functional additions relative to a deliberately minimal reference. They are not an isolated cost of reinforcement learning.

C.4 Integration Timing Inventory

Table C.4. Post-route timing checkpoints for successively integrated blocks.

Block	Minimum period (ns)	Maximum frequency (MHz)	Status
B1	6.253	159.923	Pass
B2	9.591	104.264	Pass
B3	6.517	153.445	Pass
B4	8.391	119.175	Pass
B5	9.670	103.413	Pass
B6	9.252	108.085	Pass

C.5 Physical Status Distribution and Cycle Detail

Table C.5. Status counts in the complete 84-row physical corpus. Counts are identical for H0–H4 because the replay supplies common request and candidate-validity fields.

Configuration	OK	No path	Invalid
H0	76	5	3
H1	76	5	3
H2	76	5	3
H3	76	5	3
H4	76	5	3

Table C.6. Normal-result cycle distributions at 100 MHz.

Configuration	Rows	Minimum	Mean	Median	Maximum
H0	76	2	2.000000	2	2
H1	76	7	201.763158	309	309
H2	76	8	202.763158	310	310
H3	76	8	202.763158	310	310
H4	76	10	204.763158	312	312

The minimum-to-median gap is caused by deterministic arithmetic bypass, not timing jitter. Equal objective ranges and single-feasible-candidate requests avoid one or more iterative normalizations; two nonconstant candidates exercise the full shared divider sequence.

C.6 Paired Route-Quality Detail

Table C.7. Detailed jointly successful physical comparisons. Difference is first configuration minus second in UNORM16 units.

Comparison	Pairs	Mean	95% CI	Higher	Lower	Ties	Sign p	d_z
H4–H2	58	86.465517	$[-25.121, 226.621]$	5	1	52	0.21875	0.176084
H4–H3	58	-78.051724	$[-242.281, 77.534]$	2	5	51	0.453125	-0.126716

Dividing the mean differences by 65,535 gives $+0.00131938$ and -0.00119099. The exact sign test excludes ties. The confidence limits use 10,000 nonparametric paired-row bootstrap replicates, seed 20260731, and percentile limits with linear R-7 interpolation. The executable analysis and row-level differences are included with the source.

C.7 Reward Scale and Policy Storage Detail

Table C.8. Evaluation reward extrema by outcome.

Outcome	Expression	Interpretation
Blocked, no switch	-4	Blocking term only
Blocked, switch	-4.25	Blocking plus profile-change penalty
Successful	$r_t^{\text{eval}} = 4 + 2U_Q + U_B - 0.25h - 0.25\sigma$	Bounded quality/balance utility offset by hop and switching costs
Declared maximum	6.75	Nominal best successful combination under the registered contract

Table C.8 reports the common evaluation reward. The successful training reward replaces U_B with $4U_B$; the blocked rule remains $r_t^{\text{train}} = -4 - 0.25\sigma_t$ because its balance utility is zero.

The deployed policy requires 512 useful bits (256×2). The profile ROM adds 72 useful bits (4×18). By comparison, the archival Q-table contains 16,384 bits (1024×16). Exporting the greedy actions therefore removes the runtime Q-value maximum and greatly reduces useful policy storage, while preserving one deterministic action for every quantized state.

C.8 Supporting VLSI Detail

Table C.9. Generic Yosys synthesis results for isolated kernel variants.

Variant	Implementation distinction	Generic cells	Approx. error
Interpolation V0	256-entry floor lookup	2,258	3.1853%
Interpolation V1	128-entry floor lookup	1,944	6.4597%
Interpolation V2	128-entry linear interpolation	3,399	0.0587%
LUT/interpolation arithmetic kernel (repository identifier FDPE-V3)	64-entry linear interpolation	3,014	0.1986%
SKAG-W0	Four runtime 16-by-16 products	5,343	—
SKAG-W1	Fixed-coefficient shift-add logic	503	—
Pareto-C0	Full dominance and tie-break	497	—
Pareto-C1	Score-first priority selector	472	—

The interpolation rows document a supplementary arithmetic study rather than current candidate-evaluator functionality. Approximation error is reported only for that isolated numerical experiment.

Table C.10. Simplified ngspice transmission-gate multiplexer load sweep.

Load (fF)	Average delay (ps)	Capacitive-load quantity (fJ)
5	10.38	8.10
10	17.27	16.20
20	26.64	32.40
50	54.31	81.00

The last column is the analytical $\frac{1}{2}CV^2$ value for the stated load at 1.8 V. It is not measured **QFlow** energy per route decision.

CHAPTER D
EXPERIMENTAL SCOPE AND REPRODUCIBILITY NOTES

D.1 Measurement boundaries

Physical-board evidence refers to records returned by a programmed Nexys 3 FPGA. Post-route timing and resource values come from Xilinx ISE implementation reports. RTL and software results are identified as simulations, and the OpenROAD results refer to isolated physical-design kernels. These evidence types answer different questions and are not combined into a single performance measure.

The reported kernel latency begins when a synchronous request is accepted and ends when the FPGA asserts `done`. It excludes host processing, UART serialization, JTAG programming, KMS acquisition, candidate generation, security checks, and route installation.

D.2 Repository milestone identifiers

Some repository directories retain P0–P6 prefixes because the evidence was archived in sequential engineering milestones. The directories identify the software reference and contract foundation, literature verification, policy training and freeze, controller verification, FPGA integration, and same-board replay stages described by their accompanying completion records. These identifiers are repository milestone labels and are not experimental variables. The scientific variables used in this thesis are the hardware evaluation configurations H0–H4, the research hypotheses RH1–RH5, scoring profiles Π_0–Π_3, and candidate routes $\mathcal{R}_i$.

D.3 Reproducibility records

Every headline result can be traced to a version-controlled CSV, JSON summary, implementation report, raw replay record, or physical-design report. Appendix A lists the relevant branches, commits, and paths.

CHAPTER E
WORKED CONTROLLER AND VERIFICATION CASES

This appendix applies the registered equations to concrete boundary and failure cases. The values are deterministic worked examples, not additional experimental measurements.

E.1 Threshold Equality and State Encoding

Consider the controller input

$$(x_K, x_Q, x_L, x_I) = (0.25, 0.93, 0.75, 0.125). \tag{E.1}$$

Round-half-up UNORM16 conversion produces codes $(16384, 60948, 49151, 8192)$. Every value is exactly on a registered threshold, so the left-closed boundary rule gives bins

$$(b_K, b_Q, b_L, b_I) = (1, 2, 3, 1). \tag{E.2}$$

The radix-4 address is

$$z = (((1 \times 4) + 2) \times 4 + 3) \times 4 + 1 \tag{E.3}$$

$$= 109, \tag{E.4}$$

whose binary representation is `01 10 11 01`. This one example checks the feature ordering, equality behavior, radix construction, and bit slicing.

Table E.1. Additional state examples and first-match H3 results. Values are shown as bins because quantization has already been applied.

b_K	b_Q	b_L	b_I	z	H3 action	First matching rule
0	0	3	3	15	Π_1	Critical occupancy (priority 1)
2	1	3	3	175	Π_2	Low route-quality bin (priority 2)
2	2	1	2	166	Π_1	High imbalance (priority 3)
2	2	3	1	173	Π_3	Saturated load with healthy occupancy/quality and low imbalance (priority 4)
1	2	3	1	109	Π_0	No preceding rule matches (default)

The first row also demonstrates priority: even though other stress conditions are present, $b_K = 0$ selects Π_1. The fourth row shows that the low-latency profile is requested only when

all four conjunctive conditions hold. Here “low latency” is the registered profile label for path-cost emphasis, not a promise about the number of FPGA kernel cycles.

E.2 UNORM16 Boundary Cases

The converter clips before scaling. Representative values are

Table E.2. Round-half-up and saturation examples for UNORM16.

Real input	Code	Reason
-0.10	0	Lower saturation
0	0	Exact lower endpoint
0.125	8,192	$\lfloor 0.125(65535) + 0.5 \rfloor$
0.25	16,384	First occupancy/load threshold
0.50	32,768	Middle occupancy/load threshold
0.75	49,151	Third occupancy/load threshold
0.90	58,982	First route-quality threshold
0.93	60,948	Second route-quality threshold
0.96	62,914	Third route-quality threshold
1.00	65,535	Exact upper endpoint
1.20	65,535	Upper saturation

Comparing integer codes rather than reconstructed real numbers removes ambiguity at thresholds. For example, a code of 60,948 enters route-quality bin 2, whereas 60,947 remains in bin 1.

E.3 Reward Examples

Suppose a successful route has bounded quality $b = 0.96$, post-decision imbalance $I^{\text{post}} = 0.25$, and three hops. Then

$$U_Q = \frac{0.96 - 0.90}{0.10} = 0.6, \qquad U_B = 1 - 0.25 = 0.75. \tag{E.5}$$

With no profile change, the evaluation reward is

$$r_t^{\text{eval}} = 4 + 2(0.6) + 0.75 - 0.25(3) = 5.20. \tag{E.6}$$

With a profile change, $r_t^{\text{eval}} = 4.95$. When no candidate is feasible, $r_t^{\text{eval}} = r_t^{\text{train}} = -4.00$ without a switch and -4.25 with one; the route terms are zero.

For the controlled training target, only the balance coefficient changes from one to four. The same successful non-switching transition therefore has target reward

$$r_t^{\text{train}} = 4 + 2(0.6) + 4(0.75) - 0.25(3) = 7.45. \tag{E.7}$$

The final validation table does not report this 7.45-scale quantity; it recomputes 5.20 on the original evaluation scale. This worked case illustrates why training targets and reported evaluation utilities must be named separately.

Table E.3. Direction of each term in the worked reward.

Change	Reward effect	Scope
Successful rather than blocked	$+8$ before route utilities/costs	Service outcome
Higher quality above 0.90	Positive, clipped at $+2$	Abstract experimental code
Smaller post-decision imbalance	Positive	Synthetic balance objective
One extra hop	-0.25	Topological path length
Profile change	-0.25	Controller stability

E.4 Candidate Normalization Edge Cases

For two feasible candidates, the range denominator is one greater than the raw difference. If cost codes are $98{,}304$ and $131{,}072$, the larger cost receives

$$\text{round}_{\text{half up}}\left(\frac{32768 \times 65535}{32769}\right) = 65533, \tag{E.8}$$

not 65,535. If both cost codes are equal, both normalized cost penalties are zero and the cost objective cannot distinguish the routes. The comparator then depends on the remaining profile-weighted objectives and, if the complete scores tie, the raw cost/hop/slot ordering.

With only one feasible candidate, every objective range is constant, all three normalized penalties are zero, and the candidate wins regardless of profile. This does not mean the route has perfect physical quality; it means there is no feasible alternative in the supplied set against which to normalize it. The short cycle cases in H1–H4 occur when such constant ranges bypass iterative division.

Table E.4. Deterministic winner behavior for common edge cases.

Candidate condition	Status	Selection behavior
No feasible candidate	`NO_PATH`	Path 255; score/quality zero
Exactly one feasible candidate	`OK`	Select it; normalized ranges are zero
Two candidates, unequal scores	`OK`	Select smaller 18-bit score
Equal scores, unequal raw cost	`OK`	Select smaller UQ16.16 path cost
Equal score/cost, unequal hops	`OK`	Select smaller hop count
Equal score/cost/hops	`OK`	Select smaller candidate slot
Malformed field combination	`INVALID_REQUEST`	Path 255; score/quality, state/profile zero

E.5 Dwell and Failure Sequences

The dwell mechanism is evaluated on valid accepted decisions, including valid no-path decisions. An invalid request does not create a controller decision. The illustrative sequence in Table E.5 assumes that Π_0 is active and that the three-decision dwell has just been reset.

Table E.5. Illustrative minimum-dwell sequence. Requested profiles are hypothetical policy-ROM outputs; the table demonstrates controller semantics only.

Decision	Request valid	Requested	Applied	Reason
1	Yes	Π_2	Π_0	Minimum dwell not satisfied
2	Yes, no path	Π_2	Π_0	Valid controller decision advances dwell
3	No	—	—	Invalid request rejected; dwell unchanged
4	Yes	Π_2	Π_2	Third valid dwell decision admits change
5	Yes	Π_1	Π_2	Dwell reset after the admitted change

This sequence explains why no-path rows are included in the 81 adaptive decisions while the three invalid physical records are not. It also shows why profile transition counts cannot be reconstructed merely by counting changes in raw policy ROM requests; the applied profile is a stateful result of policy and dwell.

E.6 Verification Coverage Matrix

Table E.6. Verification levels, exact covered quantities, and the defect class each level can detect.

Level	Registered coverage	Comparison rule	Primary defect class
Software regression	74 tests	Exact values and declared exceptions	Reward, quantization, packing, path-selection logic
Threshold boundaries	36 below/equal/above cases	Exact bin and state ID	Off-by-one and real-to-code boundary errors
State/policy exhaustiveness	256 state addresses and all four actions	Exact ROM word/action	Address order, missing profile, export corruption
Candidate evaluator RTL	519 of 519 vectors	Exact winner, score, and status	Divider, saturation, range, Tchebycheff, tie-break defects
Integrated policy shell	101 of 101 expected results	Exact output vector	Pipeline alignment, H0–H4 mode selection, dwell interaction
Integrated cycle tracing	2,125 cycle vectors; 530 valid decisions	Exact accepted-edge-to-done distance	Busy/reset/wait timing and counter boundary
Independent implementation	Five synthesis/map/place-route/bitstream flows	100 MHz constraint and report extraction	Configuration-specific mapping/timing defects
Physical replay	420 records; 3,780 checked fields	Exact integer equality; no tolerance	Board transport, formatter, parser, bitstream, and hardware/reference mismatch

Exactness at one level does not replace another. A Python test cannot establish post-route timing; a timing report cannot establish UART parsing; physical bit-exactness cannot establish route-quality superiority. The ladder therefore supports several bounded claims rather than one undifferentiated "validated" label.

E.7 Result-Field Accounting

There are 84 physical records per mode, nine compared fields per record, and five modes:

$$84 \times 9 = 756 \text{ fields/mode}, \qquad 5 \times 756 = 3780 \text{ fields total.} \tag{E.9}$$

The complete corpus has 76 OK, five no-path, and three invalid records per mode. Adaptive profile counts use $84 - 3 = 81$ decisions because no-path is a valid controller result. Held-out route-quality pairs use 58 rows because only jointly OK rows have a selected-route quality code. These three denominators answer different questions and should not be substituted for one another.

E.8 Interpretation Checklist

The worked cases support the following reading rules:

1. A route-quality code is dimensionless UNORM16 and larger is better; it is not stored-key coherence.
2. A smaller Tchebycheff score is better, but scores produced under different active ratios are not a common physical utility scale.
3. A no-path result is a valid evaluated request with no feasible candidate; an invalid result is a malformed request rejected by the interface contract.
4. H4's two-cycle increment over H3 is controller setup; H0's roughly 200-cycle advantage comes from bypassing the iterative multi-objective engine.
5. A mean reward near 4.67 is a dimensionless sum under the stated equation.
6. Confidence intervals crossing zero and unadjusted exploratory sign tests do not establish route-quality superiority.

REFERENCES

[1] Yuan Cao, Yongli Zhao, Qin Wang, Jie Zhang, Soon Xin Ng, and Lajos Hanzo. The evolution of quantum key distribution networks: On the road to the Qinternet. *IEEE Communications Surveys & Tutorials*, 24(2):839–894, 2022.

[2] Miralem Mehic, Marcin Niemiec, Stefan Rass, Jiajun Ma, Momtchil Peev, Alejandro Aguado, Vicente Martin, Stefan Schauer, Andreas Poppe, Christoph Pacher, and Miroslav Voznak. Quantum key distribution: A networking perspective. *ACM Computing Surveys*, 53(5):1–41, 2020.

[3] Vicente Martin, Alejandro Aguado, Juan P. Brito, David R. Lopez, Victor Lopez, Momtchil Peev, Christoph Pacher, Alberto Pastor, Juan P. B. Palomar, and Diego R. Lopez. MadQCI: A heterogeneous and scalable SDN-QKD network deployed in production facilities. *npj Quantum Information*, 10, 2024.

[4] Hao-Ze Chen et al. Implementation of carrier-grade quantum communication networks over 10000 km. *npj Quantum Information*, 11, 2025.

[5] Lin Bi, Minghui Miao, and Xiaoqiang Di. A dynamic-routing algorithm based on a virtual quantum key distribution network. *Applied Sciences*, 13(15), 2023.

[6] Evgeniy O. Kiktenko, Andrey Tayduganov, and Aleksey K. Fedorov. Routing algorithm within the multiple non-overlapping paths' approach for quantum key distribution networks. *Entropy*, 26(12), 2024.

[7] Qiaolun Zhang, Omran Ayoub, Alberto Gatto, Jun Wu, Francesco Musumeci, and Massimo Tornatore. Routing, channel, key-rate, and time-slot assignment for QKD in optical networks. *IEEE Transactions on Network and Service Management*, 21(1):148–160, 2024.

[8] Yuanchen Hao, Yuheng Xie, Wenpeng Gao, and Jianjun Tang. Q-learning for resource-aware and adaptive routing in trusted-relay QKD network. *Photonics*, 12(10), 2025.

[9] Tim Johann, Sebastian Kühl, and Stephan Pachnicke. Adaptive routing for meshed QKD networks of flexible size using deep reinforcement learning. *Photonics*, 13(2), 2026.

[10] Qing Lu, Zhenguo Lu, Hongzhao Yang, Shen-Shen Yang, and Yongmin Li. FPGA-based implementation of multidimensional reconciliation encoding in quantum key distribution. *Entropy*, 25(1), 2023.

[11] Natarajan Venkatachalam, Foram P. Shingala, C. Selvagangai, S. Hema Priya, S. Dillibabu, Pooja Chandravanshi, and Ravindra P. Singh. Scalable QKD postprocessing system with reconfigurable hardware accelerator. *IEEE Transactions on Quantum Engineering*, 4:1–14, 2023.

[12] Lianye Liao, Xinyi Wu, Ye Chen, Xiaodong Fan, Zhiyu Tian, Jinquan Huang, Tonglin Mu, Junran Guo, Minjie Liu, Bo Liu, and Shihai Sun. Efficient FPGA implementation of polar codes-based information reconciliation for quantum key distribution. *Scientific Reports*, 15, 2025.

[13] Nishanth Chandra and Pradeep Kumar Krishnamurthy. FPGA-based synchronization of frequency-domain interferometer for QKD. *IEEE Transactions on Quantum Engineering*, 6:1–10, 2025. IEEE document 10769019.

[14] Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.

[15] Sandra Zajac and Sandra Huber. Objectives and methods in multi-objective routing problems: A survey and classification scheme. *European Journal of Operational Research*, 290(1):1–25, 2021.

[16] Refael Hassin. Approximation schemes for the restricted shortest path problem. *Mathematics of Operations Research*, 17(1):36–42, 1992.

[17] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[18] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[19] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, 2 edition, 2018.

[20] Li-Quan Chen, Jing-Qi Chen, Qian-Ye Chen, and Yong-Li Zhao. A quantum key distribution routing scheme for hybrid-trusted QKD network system. *Quantum Information Processing*, 22, 2023.

[21] Md Shahbaz Akhtar, Krishnakumar G., Vishnu B., and Abhishek Sinha. Fast and secure routing algorithms for quantum key distribution networks. *IEEE/ACM Transactions on Networking*, 31(5):2281–2296, 2023.

[22] Vimal Bhatia, Adolph Kasegenya, and Bowen Chen. Dynamic security-aware resource allocation in quantum key distribution-enabled optical networks. *Photonics*, 12(7), 2025.

[23] Peng Yong Kong. Routing with minimum activated trusted nodes in quantum key distribution networks for secure communications. *IEEE Internet of Things Journal*, 11(9):15219–15228, 2024.

[24] Alin-Bogdan Popa and Pantelimon George Popescu. Optimal key forwarding strategy in QKD behaviours. *Scientific Reports*, 14, 2024.

[25] Mengyao Li, Qiaolun Zhang, Alberto Gatto, Stefano Bregni, Giacomo Verticale, and Massimo Tornatore. DRL-based progressive recovery for quantum-key-distribution networks. *Journal of Optical Communications and Networking*, 16(9):E36–E47, 2024.

[26] Esraa M. Ghourab, Mohamed Azab, and Denis Gračanin. A quantum key distribution routing scheme for a zero-trust QKD network system: A moving target defense approach. *Big Data and Cognitive Computing*, 9(4), 2025.

[27] Purva Sharma, Shubham Gupta, Vimal Bhatia, and Shashi Prakash. Deep-reinforcement-learning-based routing for quantum key distribution networks. *IET Quantum Communication*, 4:136–145, 2023.

[28] Diego Medeiros de Abreu and Antônio Abelém. qRL: Reinforcement learning routing for quantum entanglement networks. In *2024 IEEE Symposium on Computers and Communications*, 2024.

[29] Linh Le, Tu N. Nguyen, Ahyoung Lee, and Braulio Dumba. Entanglement routing for quantum networks: A deep reinforcement learning approach. In *ICC 2022 – IEEE International Conference on Communications*, pages 395–400, 2022.

[30] Tobias Meuser, Jannis Weil, Aninda Lahiri, and Marius Paraschiv. RELiQ: Scalable entanglement routing via reinforcement learning in quantum networks. *IEEE Transactions on Communications*, 74:1860–1875, 2026.

[31] Kaijie Wei, Devanshu Garg, Ryutaro Nagai, Takao Tomono, and Hideharu Amano. FPT-EMS: An FPGA implementation using NB-LDPC code for continuous-variable quantum key distribution. In *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*, pages 117–125, 2025.

[32] Shenshen Yang, Zhilei Yan, Hongzhao Yang, Qing Lu, Zhenguo Lu, Liuyong Cheng, Xiangyang Miao, and Yongmin Li. Information reconciliation of continuous-variables quantum key distribution: Principles, implementations and applications. *EPJ Quantum Technology*, 10, 2023.

[33] Beatrice Branchini, Davide Conficconi, Donatella Sciuto, and Marco D. Santambrogio. The hitchhiker's guide to FPGA-accelerated quantum error correction. In *2023 IEEE International Conference on Quantum Computing and Engineering*, pages 338–339, 2023. IEEE document 10313923.

[34] Aravind Aji, Kurunandan Jain, and Prabhakar Krishnan. A survey of quantum key distribution (QKD) network simulation platforms. In *2021 2nd Global Conference for Advancement in Technology*, 2021.

[35] Tim Coopmans, Robert Knegjens, Axel Dahlberg, David Maier, Loek Nijsten, Julio de Oliveira Filho, Martijn Papendrecht, Julian Rabbie, Filip Rozpędek, Matthew Skrzypczyk, Leon Wubben, Walter de Jong, Damian Podareanu, Ariana Torres-Knoop, David Elkouss, and Stephanie Wehner. NetSquid, a NETwork simulator for QUantum information using discrete events. *Communications Physics*, 4, 2021.

[36] Niansong Zhang, Xiang Chen, and Nachiket Kapre. RapidLayout: Fast hard block placement of FPGA-optimized systolic arrays using evolutionary algorithms. *ACM Transactions on Reconfigurable Technology and Systems*, 2022.

[37] Barry Shackleford et al. A high-performance, pipelined, FPGA-based genetic algorithm machine. *Genetic Programming and Evolvable Machines*, 2(1):33–60, 2001.

[38] W. Tang and L. Yip. Hardware implementation of genetic algorithms using FPGA. In *Proceedings of the 47th Midwest Symposium on Circuits and Systems*, volume 1, pages I–549–I–552, 2004.

[39] Pradeep R. Fernando, Srinivas Katkoori, Didier Keymeulen, Ricardo Zebulum, and Adrian Stoica. Customizable FPGA intellectual property core implementation of a general-purpose genetic algorithm engine. *IEEE Transactions on Evolutionary Computation*, 14(1):133–149, 2010.

[40] Liucheng Guo, David B. Thomas, and Wayne Luk. Automated framework for general-purpose genetic algorithms in FPGAs. In *Applications of Evolutionary Computation*, volume 8602 of *Lecture Notes in Computer Science*, pages 714–725. Springer, 2014.

[41] Murat Peker. A fully customizable hardware implementation for general purpose genetic algorithms. *Applied Soft Computing*, 62:1066–1076, 2018.